BSIDES Prague & Budapest 2026

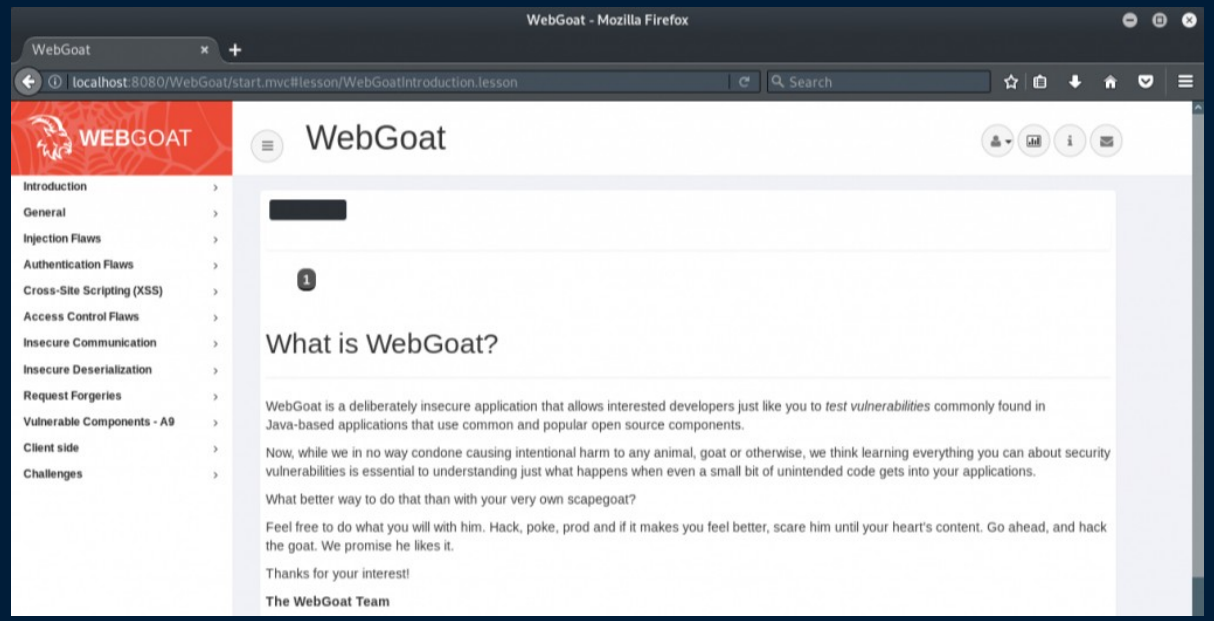
Unveiling the obscurity: Decrypting Agent-Server Communications

Jorge Escabias

Agenda

- Introduction
- Basics of Agent-Server Communication
- Tools
- Hands-on: Basic Examples
  - HTTP & HTTPs
  - TCP & TLS over TCP
  - Mutual TLS
- Diving deeper: Intercepting with Frida
- Decision Tree Mindmap
- Real World Example – Sliver C2
- Wrapping Up
- Challenge
- References

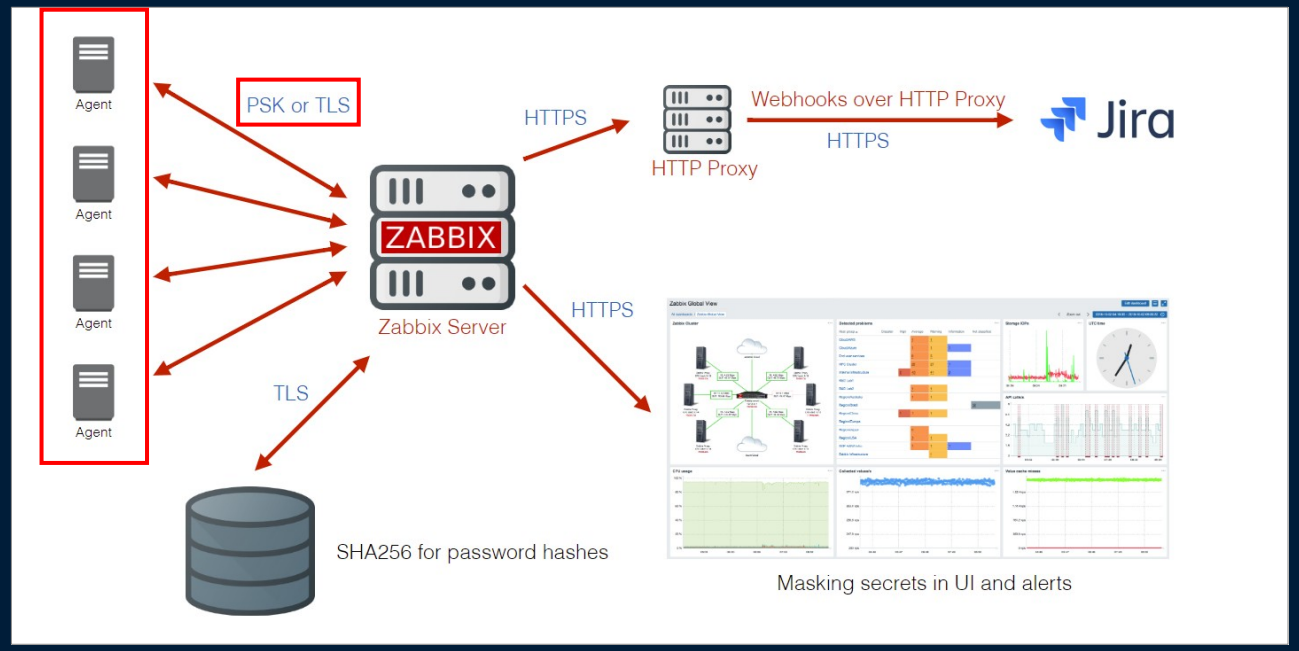
Introduction

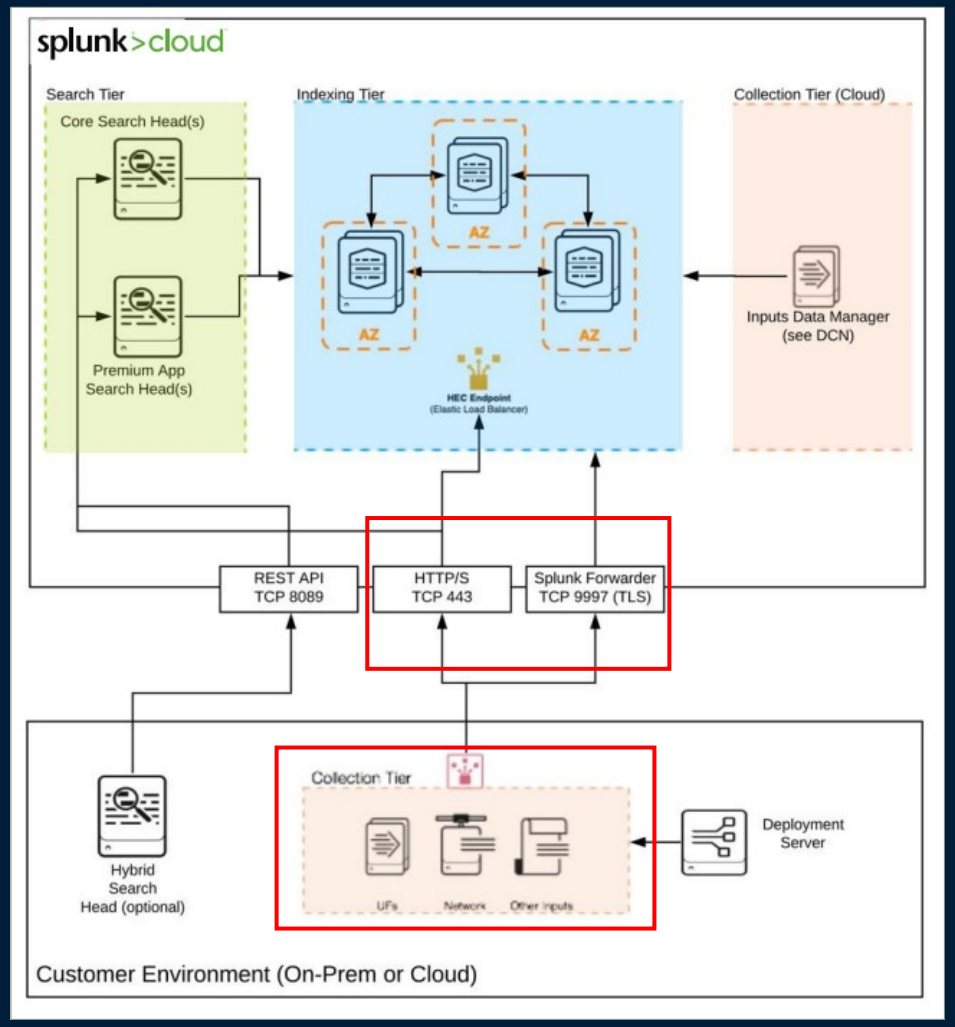


Microsoft
Active Directory

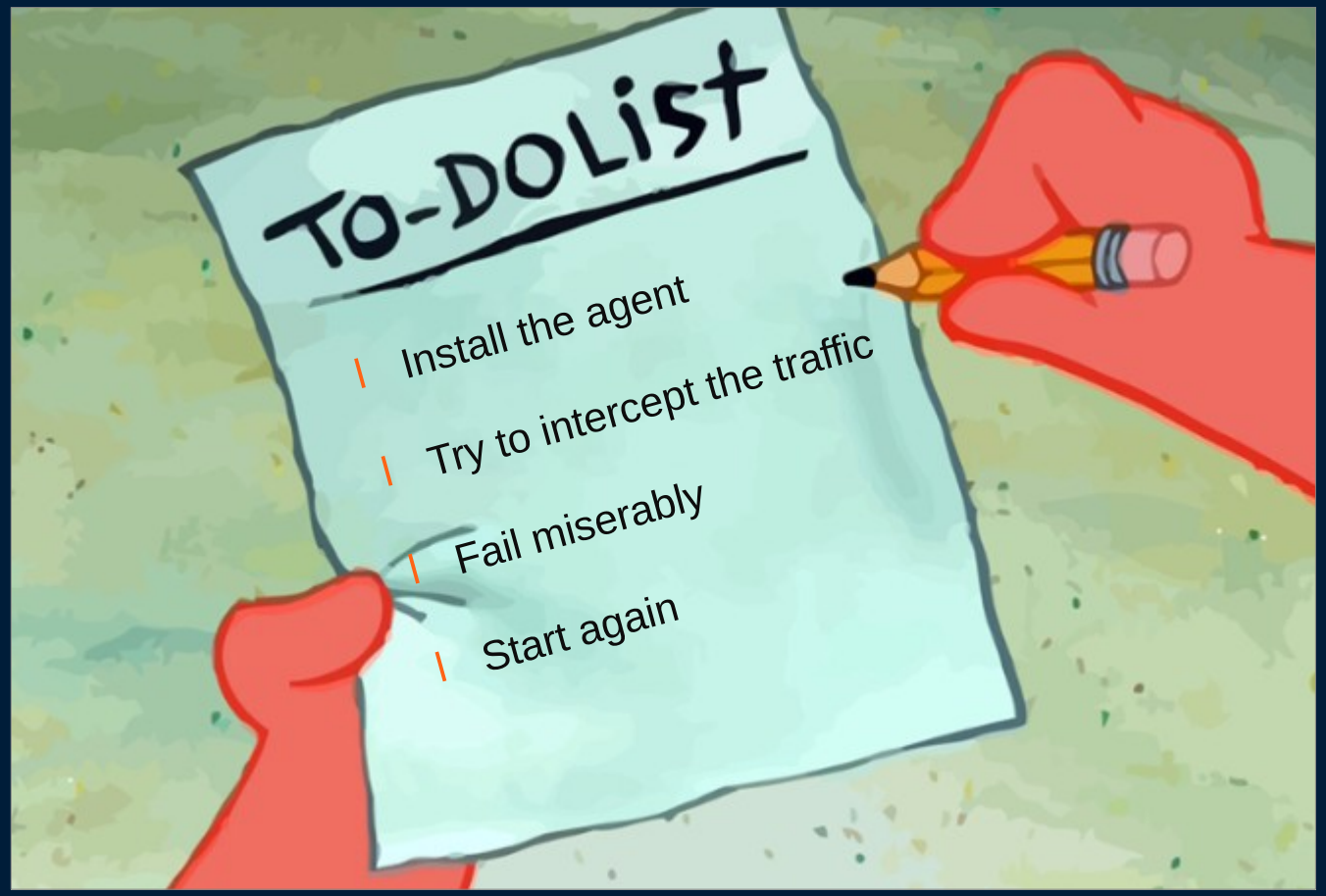
Introduction

Introduction

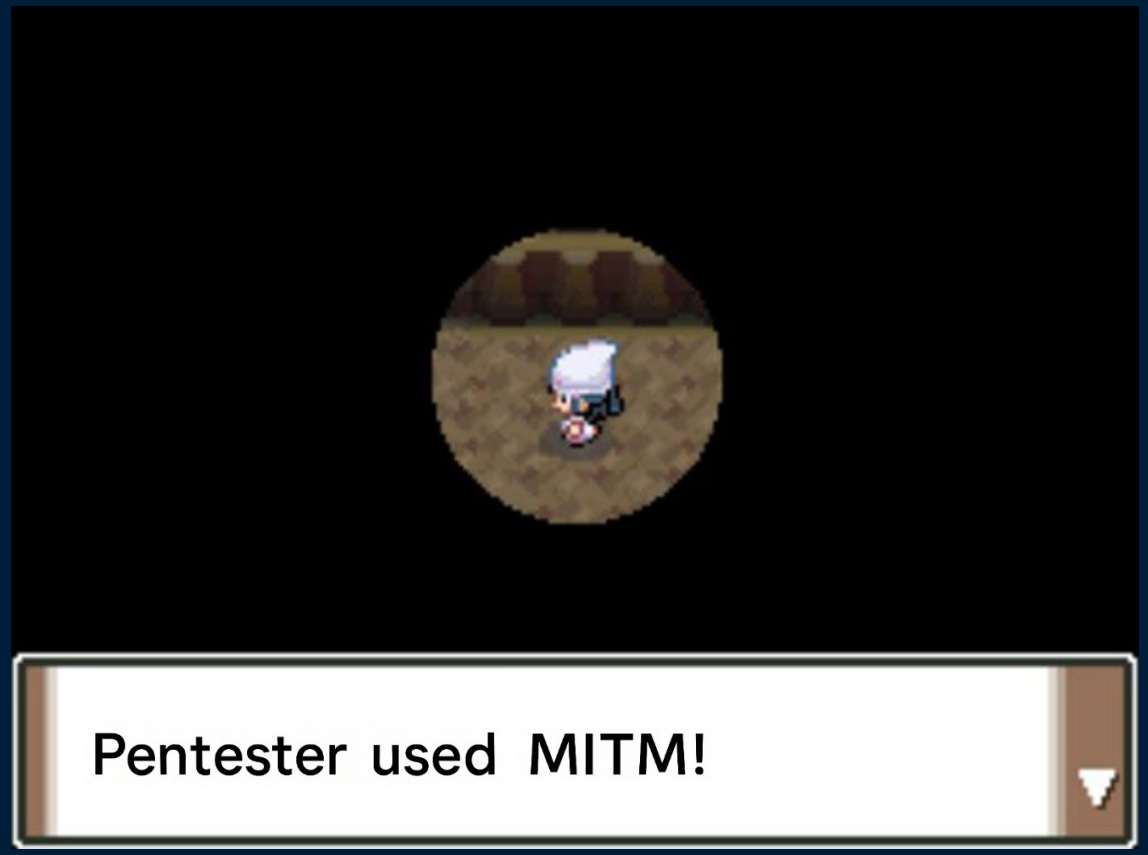


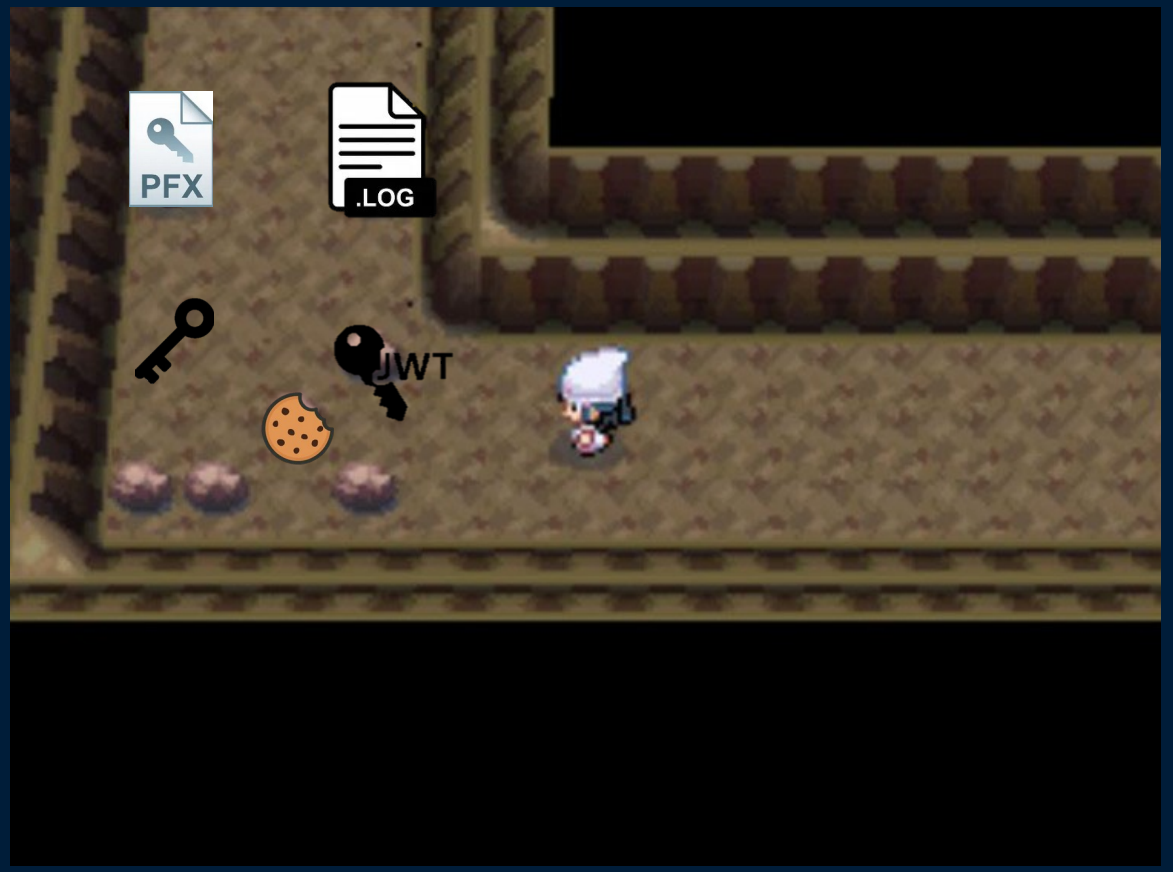


Introduction



Introduction





Basics of Agent-Server Communication

- Agents are installed on endpoints (VMs, servers, workstations)
- Sometimes, they run with high privileges (but not necessarily)
- We can separate them into two main groups:
  - Agents executing tasks communicated by the server (on-demand/scheduled). Example: Forensics tools
  - Agents collecting and sending data to the server. Example: Splunk Agent
- Various protocols are used for the communication
  - Protobuf
  - MQTT
  - gRPC
  - Raw sockets
  - Web sockets
  - …
- However, we will group them based on the layer:
  - Application Layer - HTTP
  - Transport Layer - TCP

Basics of Agent-Server Communication

- Encryption is always a problem
- Based on the agent, we might face different secure protocols
- For Application Layer
  - HTTPs
  - HTTPs with payload encryption
- For Transport Layer
  - TLS over TCP
  - TLS over TCP with payload encryption
  - mutual TLS over TCP (with payload encryption too)
- Sometimes, we might face situations where agents have some sort of SSL Pinning/SSL verification.
- Although we will not be discussing SSL Pinning, we will look at SSL Verification as an edge case

Tools

- We will use:
  - WireShark for traffic analysis (TCP or HTTP)
  - Burp Suite for HTTP and HTTPs interception
  - MITM Proxy for TCP, TLS over TCP and mutual TLS interception
  - Frida for edge cases (SSL Pinning, SSL Verification, encrypted payloads…)
  - API Monitor for supporting Frida usage
- There are many tools to do the same like:
  - Charles Proxy
  - Fiddler
  - Owasp ZAP
  - Burp Suite NoPe Extension
  - Proxifier
  - WinDBG
  - …

Lab Examples

- For each lab, we will have an agent and a server using one of the discussed protocols
- The agent will send a message to the server
- Our goal will be to intercept that message and modify it using the tools discussed before
- Depending on the situation, we may need to use one approach or another

C:\Windows\System32\cmd.exe

```
C:\Users\Ruperto\Desktop\Research Binaries\http>client.exe 192.168.233.165
Server response: HTTP/1.1 200 OK
Content-Length: 17

Hi From Server!
```

C:\Windows\System32\cmd.e

```
Message Received: POST / HTTP/1.1
Host: 127.0.0.1
Content-Type: text/plain
Content-Length: 22
Connection: close

Hello, I'm the agent!
```

Scenario 1 – Intercepting HTTP

- Intercepting HTTP is straightforward
- Wireshark can be used to identify the protocol and the type of encryption
- Using Burp Suite, we can easily intercept the traffic and even modify its content

- Situations we might face:
  - Case 1: The agent is proxy aware. Straight MITM.
  - Case 2: The agent is not proxy aware.
    - The configuration is set using a file.
    - Hardcoded configuration that cannot be changed.

Scenario 1 – Lab Example 1

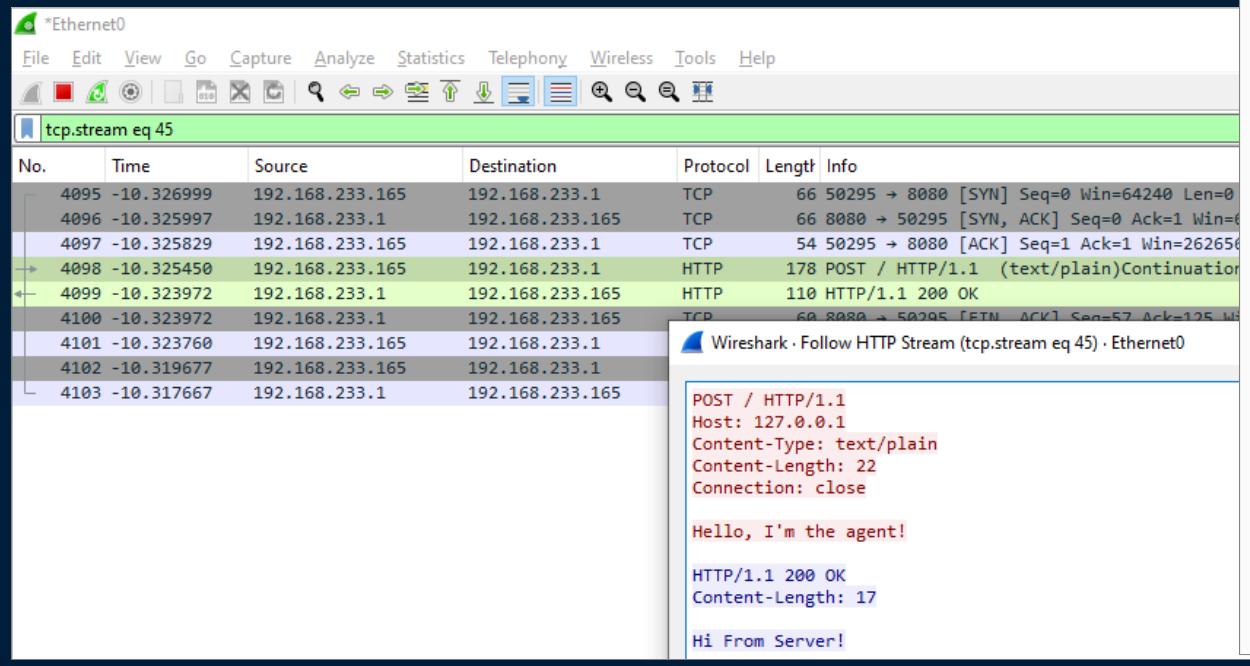
- ***Client.exe*** connects to ***Server.exe*** based on the IP provided

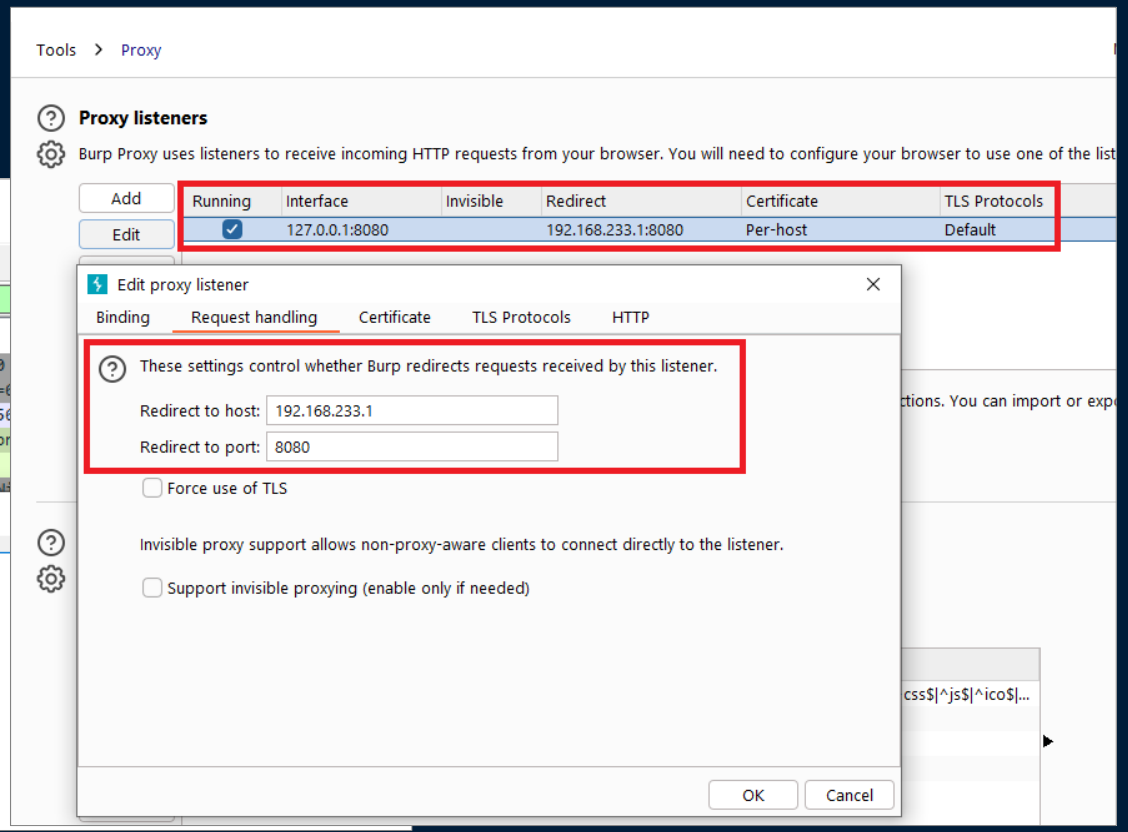
```
C:\Users\Ruperto\Desktop\Research Binaries\http>client.exe 192.168.233.1
Server response: HTTP/1.1 200 OK
Content-Length: 17

Hi From Server!
```

```
C:\Users\Jorge\Downloads\TEST>server.exe
Server listening on port 8080...
Message Received: POST / HTTP/1.1
Host: 127.0.0.1
Content-Type: text/plain
Content-Length: 22
Connection: close

Hello, I'm the agent!
```

Scenario 1 – Lab Example 1

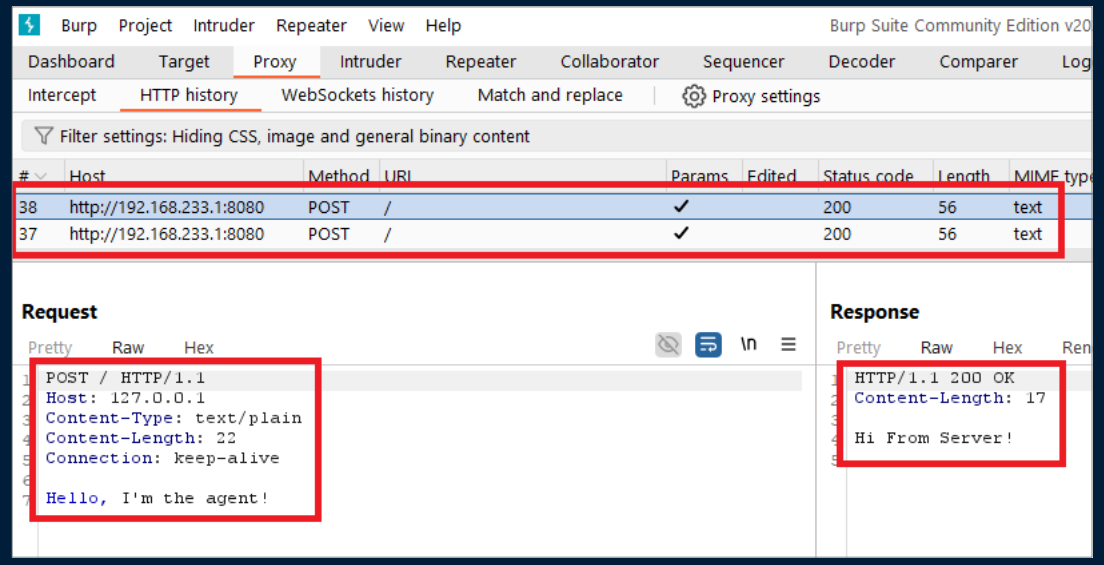
- As traffic is cleartext, we can see what is been transmitted

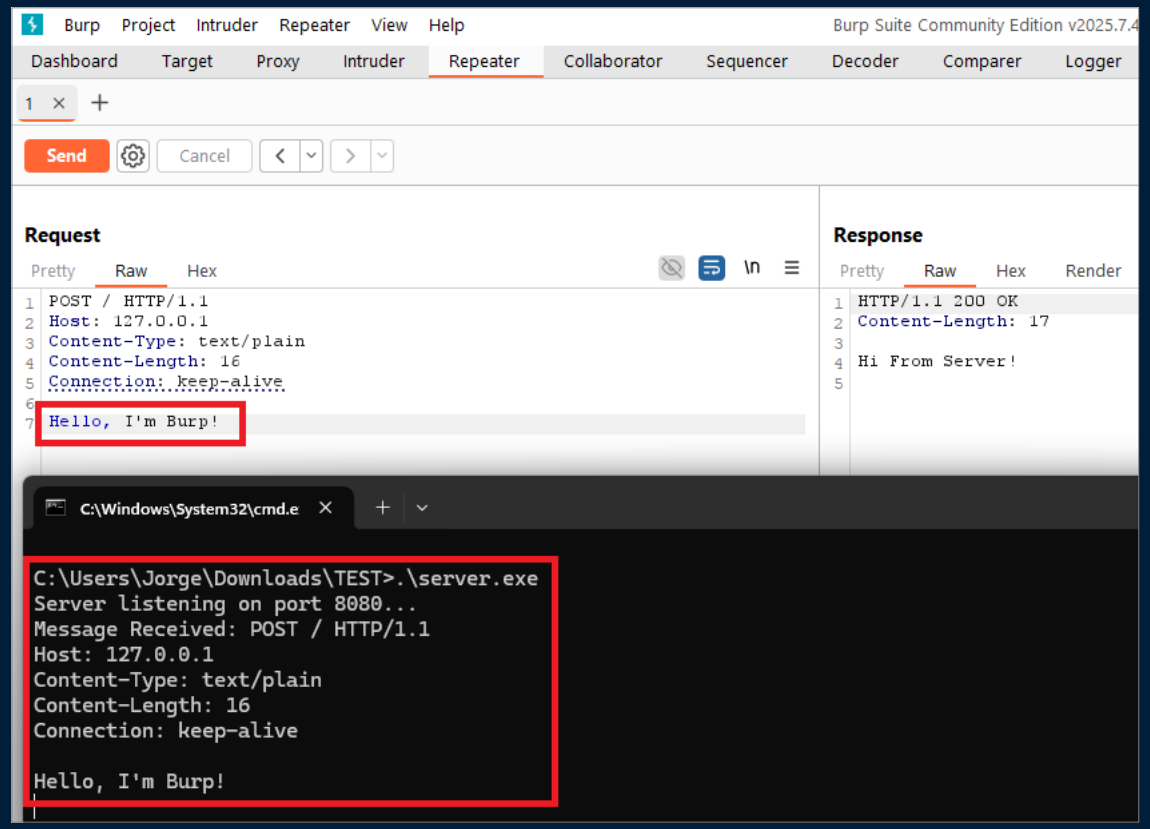




Scenario 1 – Lab Example 1

- Traffic can be intercepted and even replicated





Scenario 1 – Lab Example 2

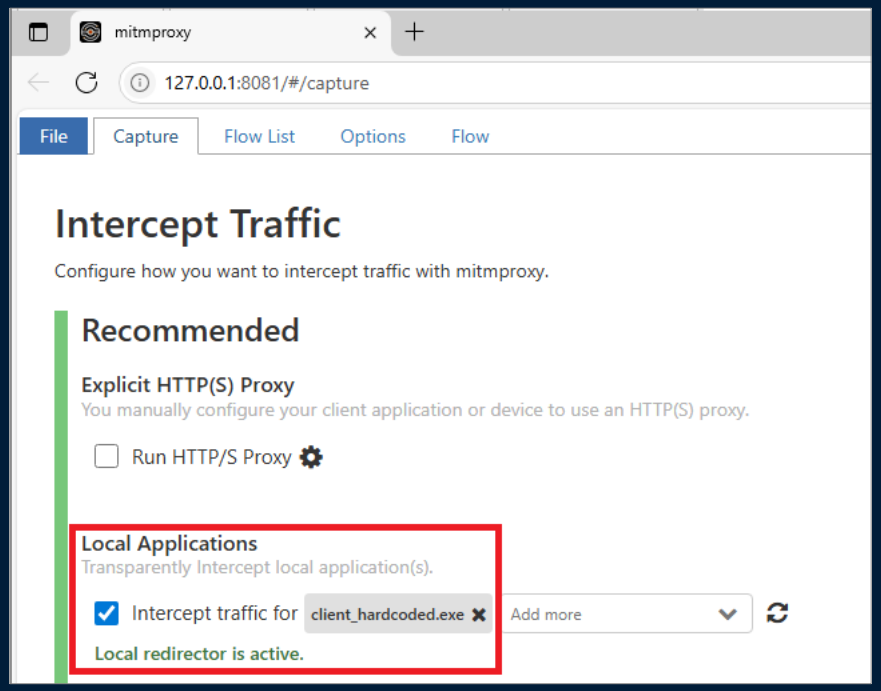
- ***Client_hardcoded.exe*** connects to ***Server.exe*** using a hardcoded IP

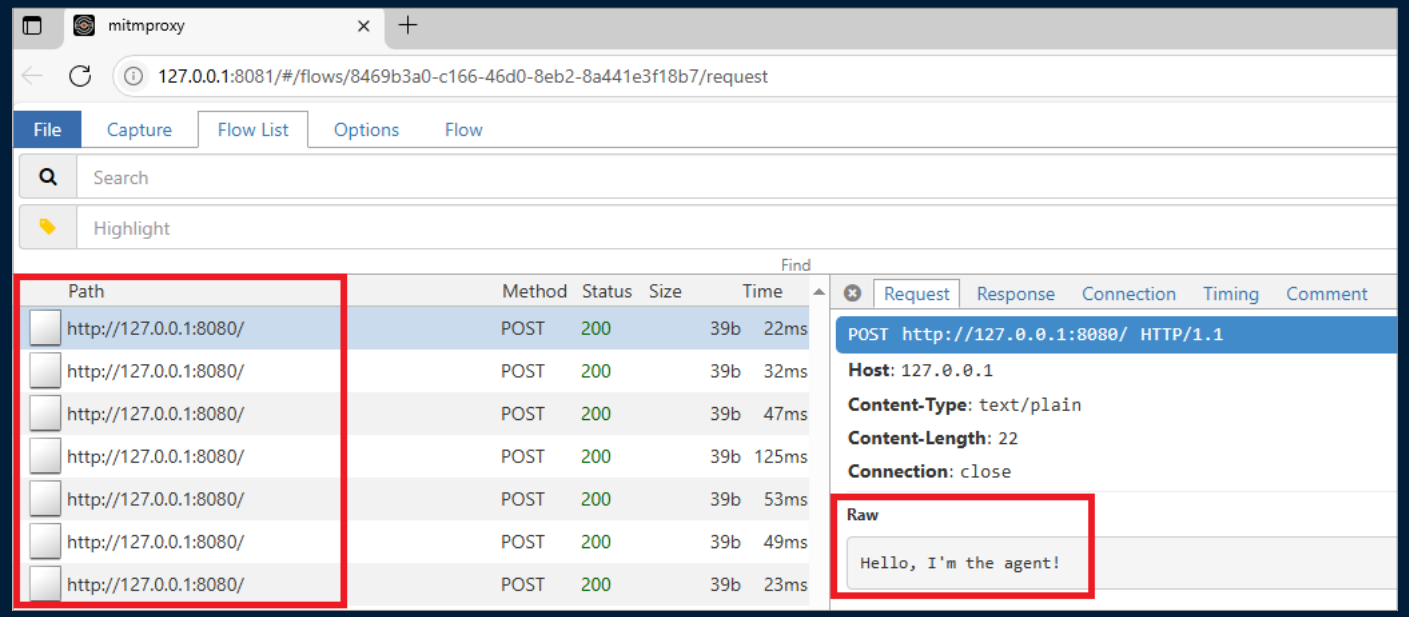
```
C:\Users\Ruperto\Desktop\Research Binaries\http>client_hardcoded.exe
Server response: HTTP/1.1 200 OK
Content-Length: 17

Hi From Server!
```

```
C:\Users\Jorge\Downloads\TEST>server.exe
Server listening on port 8080...
Message Received: POST / HTTP/1.1
Host: 127.0.0.1
Content-Type: text/plain
Content-Length: 22
Connection: close

Hello, I'm the agent with the hardcoded IP!
```

Scenario 1 – Lab Example 2

- As Burp doesn't work, **hooking** it's our only option using MITM Proxy





Scenario 2 – Intercepting HTTPs

- Intercepting HTTPs should be straightforward
- Wireshark can be used to decrypt HTTPs traffic too. However, it relays on the use of specific libraries by the agent (BoringSSL/LibreSSL).
- Using Burp Suite, we can easily intercept the traffic and even modify its content
- If not, MITM Proxy is also an alternative solution

- Situations we might face:
  - Case 1: The agent is proxy aware (Burp)
  - Case 2: The agent is **not** proxy aware but doesn't validate the certificate (Burp + invisible proxy enabled)
  - Case 3: The agent is **not** proxy aware but does validate the certificate (Burp + invisible proxy enabled + Burp's CA installed)

Scenario 2 – Lab Example 1

- Case 1: ***Proxy_aware_agent.py*** connects to ***Server.py*** using a hardcoded hostname

```
C:\Users\Jorge\Downloads\TEST\server.py:24: DeprecationWarning:
  httpd.socket = ssl.wrap_socket(
Serving HTTPS on 0.0.0.0:8443
Received POST:
Hello server, I'm using the manual proxy from Windows Settings!
```

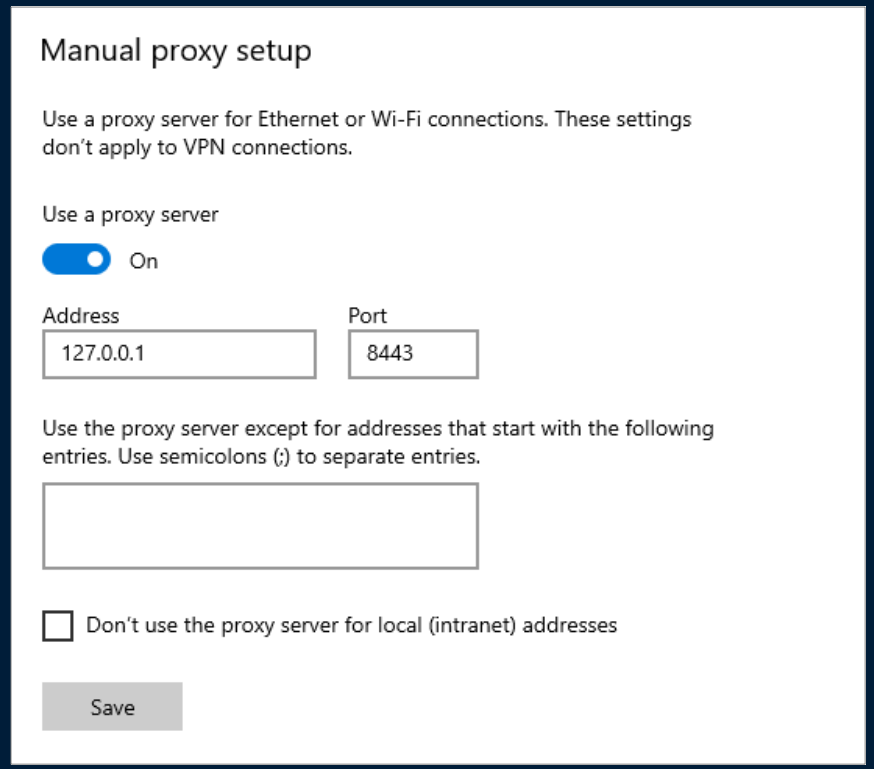
```
C:\Users\Ruperto\Desktop\Research Binaries\https>python proxy_aware_agent.py
[i] Proxy server raw value: 127.0.0.1:8443
[i] Applying proxies: {'http': '127.0.0.1:8443', 'https': '127.0.0.1:8443'}
Connection OK (status: 200 )
Hello agent, I'm your server!.
```

Scenario 2 – Lab Example 1

- Solution:
  - Enable manual proxy setup on Windows and point it to Burp

client.c | server.c | client_hardcoded.c | hosts | no_validation_no_proxy_agent.c

```
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#      102.54.94.97     rhino.acme.com          # source server
#       38.25.63.10     x.acme.com              # x client host

# localhost name resolution is handled within DNS itself.
#   127.0.0.1       localhost
#   ::1             localhost
192.168.152.100 kali
192.168.233.1 decrypting.lab
```



Scenario 2 – Lab Example 1

- After that, traffic can be intercepted easily

Request

Pretty Raw Hex

```
POST / HTTP/1.1
Accept-Encoding: gzip, deflate, br
Content-Length: 63
Host: decrypting.lab:8443
User-Agent: Python-urllib/3.9
Content-Type: text/plain; charset=utf-8
Connection: keep-alive

Hello server, I'm using the manual proxy from Windows Settings!
```

Response

Pretty Raw Hex Render

```
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.11.9
Date: Mon, 25 Aug 2025 11:18:01 GMT
Content-Type: text/plain

Hello agent, I'm your server!.
```

Scenario 2 – Lab Example 2

- Case 2: **Agent without Certificate Validation and not proxy aware** connects to ***Server.py*** using a hardcoded hostname

```
C:\Users\Jorge\Downloads\TEST>python server.py
C:\Users\Jorge\Downloads\TEST\server.py:24: DeprecationWarning: ssl.wrap_socket() is deprec
  httpd.socket = ssl.wrap_socket(
Serving HTTPS on 0.0.0.0:8443
Received POST:
Hello server, I'm the agent. I'm not validating the certificate and I'm not proxy aware!
```

```
C:\Users\Ruperto\Desktop\Research Binaries\https>no_validation_no_proxy_agent.exe
Server response:
Hello agent, I'm your server!.
```

Scenario 2 – Lab Example 2

- Solution:
  - Point etc/host to Burp's IP
  - Set Burp Listener to the server's IP enabling invisible proxy and use of TLS

C:\Users\Ruperto\Desktop\Research Binaries\https\proxy_aware_agent.py - Notepad++

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

hosts | proxy_aware_agent.py

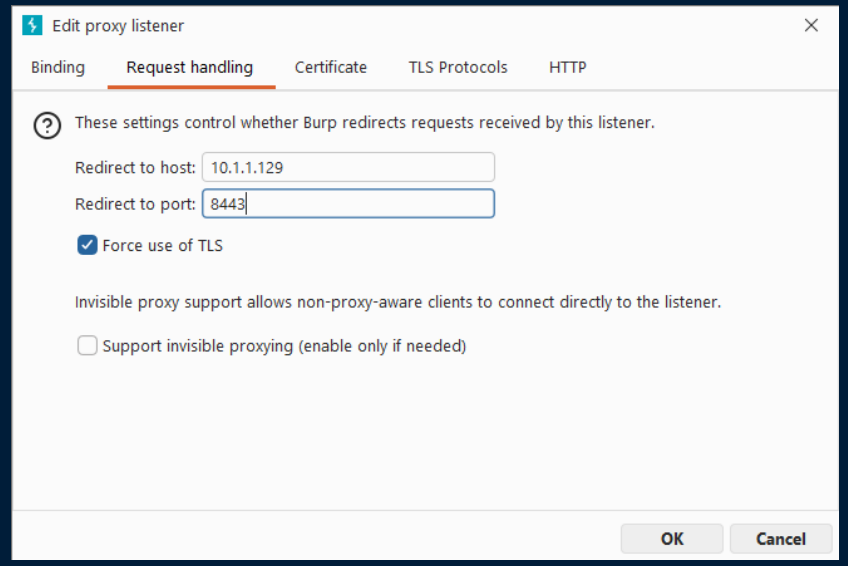
```python
import urllib.request
import ssl
import winreg

# Disable TLS verification
ssl._create_default_https_context = ssl._create_unverified_context

URL = "https://decrypting.lab:8443/"
TIMEOUT = 10
PAYLOAD = "Hello server, I'm using the manual proxy from Windows Settings!"
data = PAYLOAD.encode("utf-8")
```

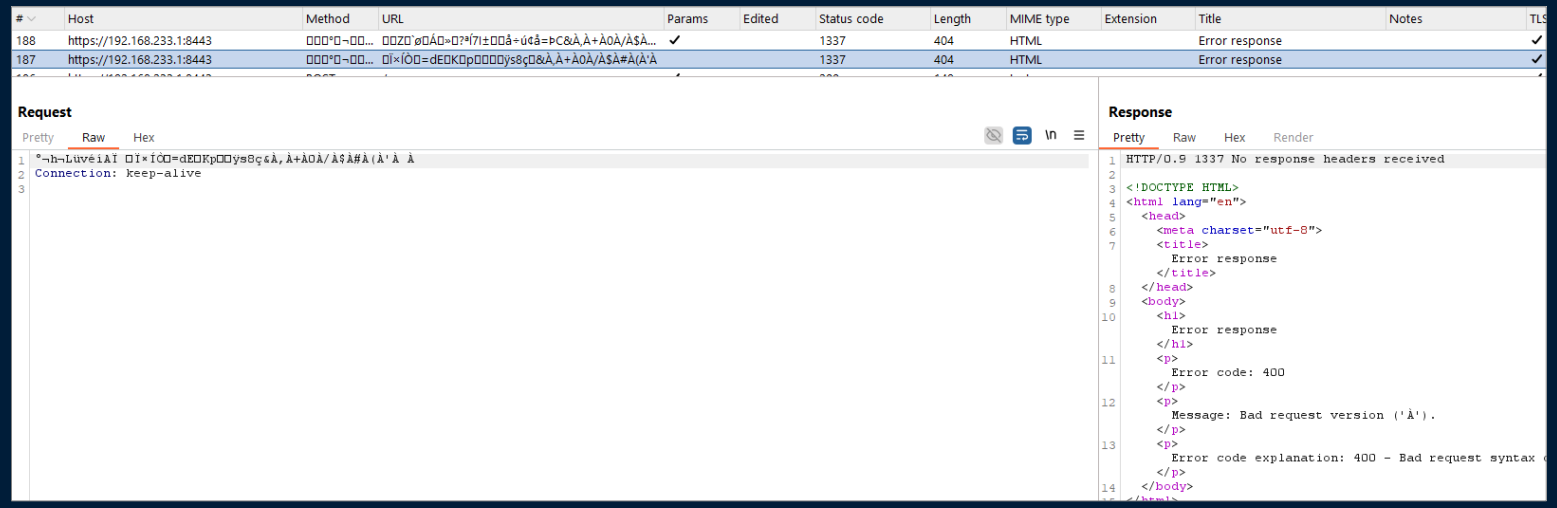
client.c | server.c | client_hardcoded.c | hosts | no_validation_no_proxy_agent.c

```
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#      102.54.94.97     rhino.acme.com          # source server
#       38.25.63.10     x.acme.com              # x client host

# localhost name resolution is handled within DNS itself.
#   127.0.0.1       localhost
#   ::1             localhost
192.168.152.100 kali
192.168.233.1 decrypting.lab
```

Scenario 2 – Lab Example 2

- If **Support Invisible Proxy** is not set, we will see something like the following:





Scenario 2 – Lab Example 2

- But, once is enabled:

Request

Pretty Raw Hex

```
1 POST / HTTP/1.1
2 Connection: keep-alive
3 Content-Type: text/plain
4 User-Agent: WinHTTPSClient/1.0
5 Content-Length: 88
6 Host: decrypting.lab:8443
7
8 Hello server, I'm the agent. I'm not validating the certificate and I'm not proxy aware!
```

Response

Pretty Raw Hex Render

```
1 HTTP/1.0 200 OK
2 Server: BaseHTTP/0.6 Python/3.11.9
3 Date: Mon, 25 Aug 2025 11:42:56 GMT
4 Content-Type: text/plain
5
6 Hello agent, I'm your server!.
7
```

Scenario 2 – Lab Example 3

- Case 3: **Agent with Certificate Validation and not proxy aware** connects to ***Server.py*** using a hardcoded hostname

```
C:\Users\Jorge\Downloads\TEST\server.py:24: DeprecationWarning: ssl.wrap_socket() is
  httpd.socket = ssl.wrap_socket(
Serving HTTPS on 0.0.0.0:8443
Received POST:
Hello server, I'm the agent. I'm validating the certificate and I'm not proxy aware!
```

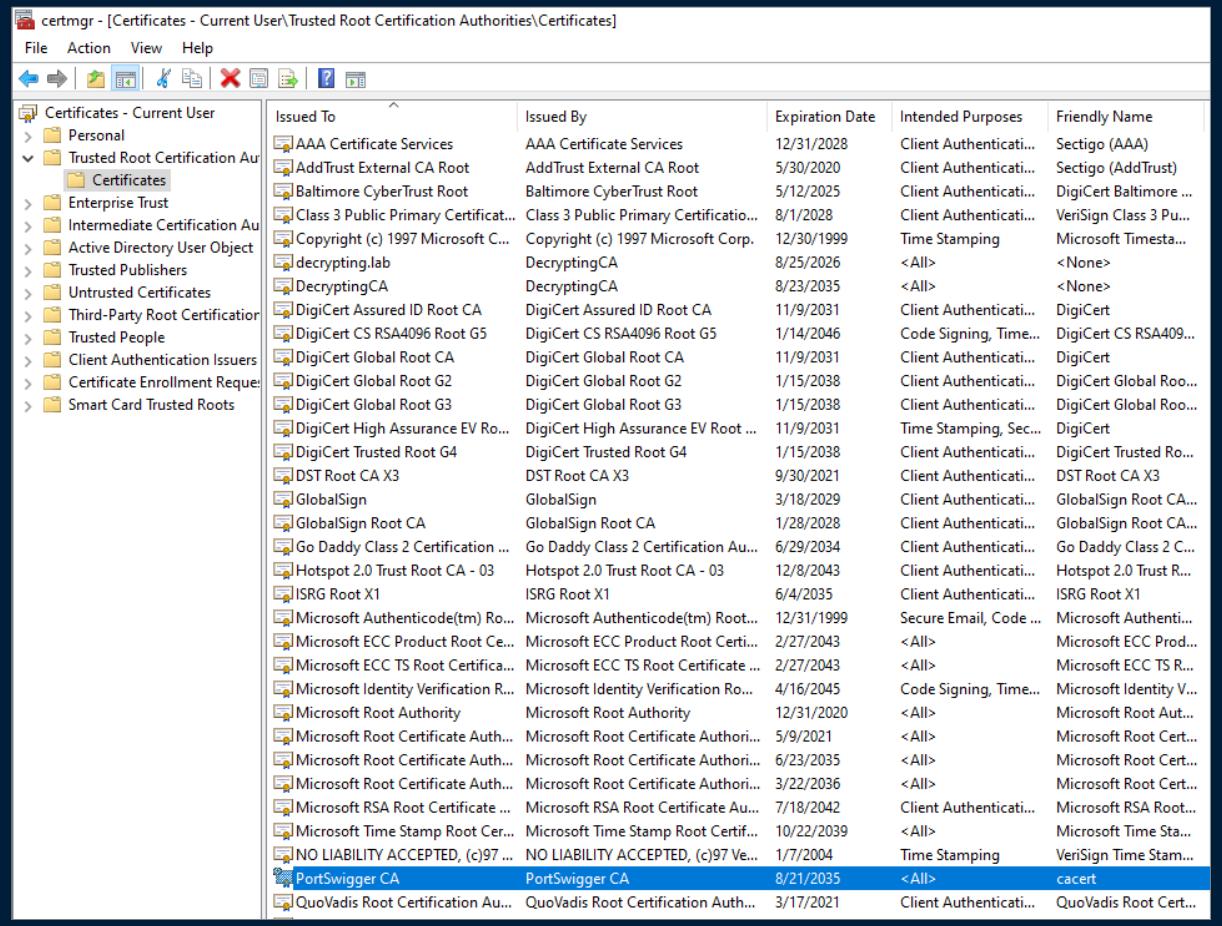
```
C:\Users\Ruperto\Desktop\Research Binaries\https>validation_and_no_proxy_agent.exe
Server response:
Hello agent, I'm your server!.
```

Scenario 2 – Lab Example 3

- Solution:
  - Point the hostname to Burp's IP by editing /etc/hosts
  - Set Burp Listener to the server's IP enabling invisible proxy and use of TLS
  - Add Burp's Certificate to Trusted Certificate Authorities

client.c | server.c | client_hardcoded.c | hosts

```
192.168.152.100 kali
127.0.0.1 decrypting.lab
```



Scenario 2 – Lab Example 3

- If Burp CA is not trusted, we will see something like this

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies
190	https://192.168.233.1:8443	□□□°□¬□□...	À			1337	457	HTML		Error response		✓	192.168.233.1	

Request

Pretty Raw Hex

```
°¬h¬N3N.□CÀG¯□ã÷KnR]GÅO¥p□]Ã&À,À+ÀOÀ/À$À#À(À'À À ÀÀ□□=<5/
Connection: keep-alive
```

Response

Pretty Raw Hex Render

```
HTTP/0.9 1337 No response headers received

<!DOCTYPE HTML>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>
      Error response
    </title>
  </head>
  <body>
    <h1>
      Error response
    </h1>
    <p>
      Error code: 400
    </p>
    <p>
      Message: Bad request version ('À\x14À\x13\x00\x9d\x00\x9c\x00=\x00&lt;\x005\x00/\x00').
    </p>
    <p>
      Error code explanation: 400 - Bad request syntax or unsupported method.
    </p>
  </body>
```

```
C:\Users\Ruperto\Desktop\Research Binaries\https>validation_and_no_proxy_agent.exe
WinHttpSendRequest failed: 12175
```

Event log

Filter Critical Error Info Debug

Time	Type	Source	Message
04:56:11 25 Aug 2025	Error	Proxy	The client failed to negotiate a TLS connection to 192.168.233.1:8443: (certificate_unknown) Received fatal alert: certificate_unknown

Scenario 2 – Lab Example 3

- Once It’s trusted, it should work:

Request

Pretty Raw Hex

```
POST / HTTP/1.1
Connection: keep-alive
Content-Type: text/plain
User-Agent: WinHTTPSClient/1.0
Content-Length: 84
Host: decrypting.lab:8443

Hello server, I'm the agent. I'm validating the certificate and I'm not proxy aware!
```

Response

Pretty Raw Hex Render

```
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.11.9
Date: Mon, 25 Aug 2025 12:52:56 GMT
Content-Type: text/plain

Hello agent, I'm your server!.
```

Scenario 3 – Intercepting TCP

- Intercepting TCP is like intercepting HTTP.
- Wireshark can be used to confirm the use of TCP.
- Burp Suite is not the best for intercepting TCP. However, we can use NoPe Extension if wanted.
- MITM Proxy is going to be our alternative here

- Situations we might face (same as HTTP):
  - Case 1: The agent is proxy aware. Straight MITM.
  - Case 2: The agent is not proxy aware.
    - The configuration is set using a file.
    - Hardcoded configuration that cannot be changed.

Scenario 3 – Lab Example 1

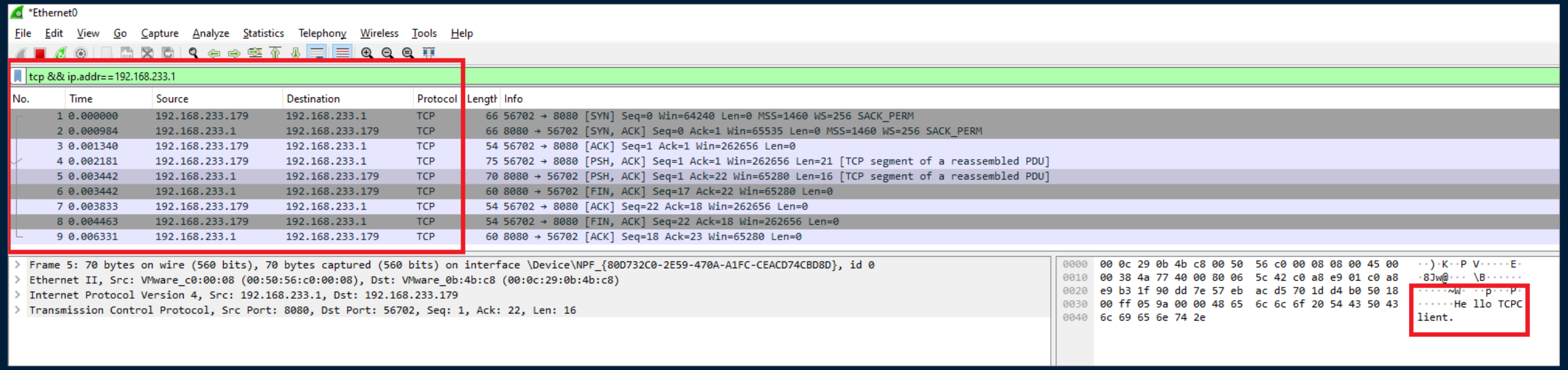
- ***TCPClient.exe*** connects to ***TCPServer.exe*** based on the IP provided

```
C:\Users\Ruperto\Desktop\Research Binaries\tcp>tcp_client.exe 192.168.233.1
Server says: Hello TCPClient.
```

```
C:\Users\Jorge\Downloads\TEST>tcp_server.exe
TCP Server listening on port 8080...
Received: Hello from TCPclient!
```

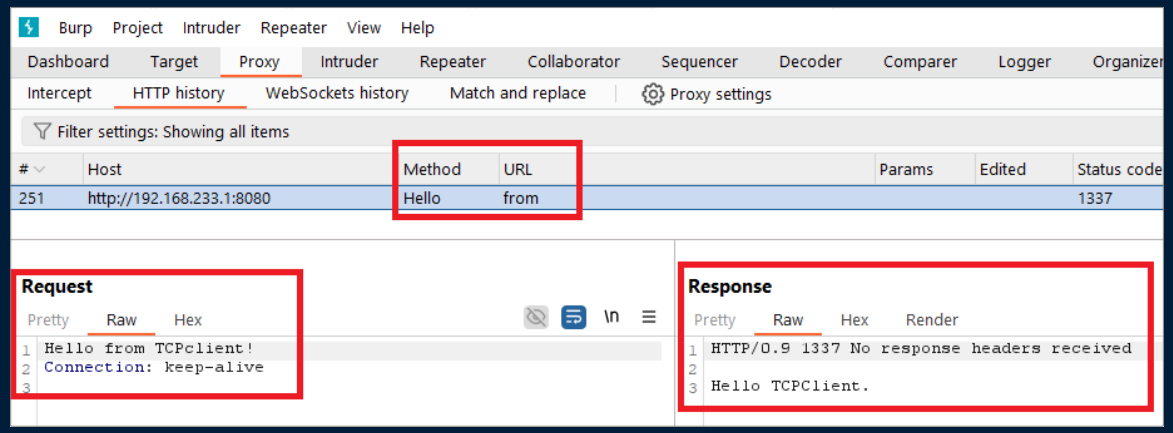
Scenario 3 – Lab Example 1

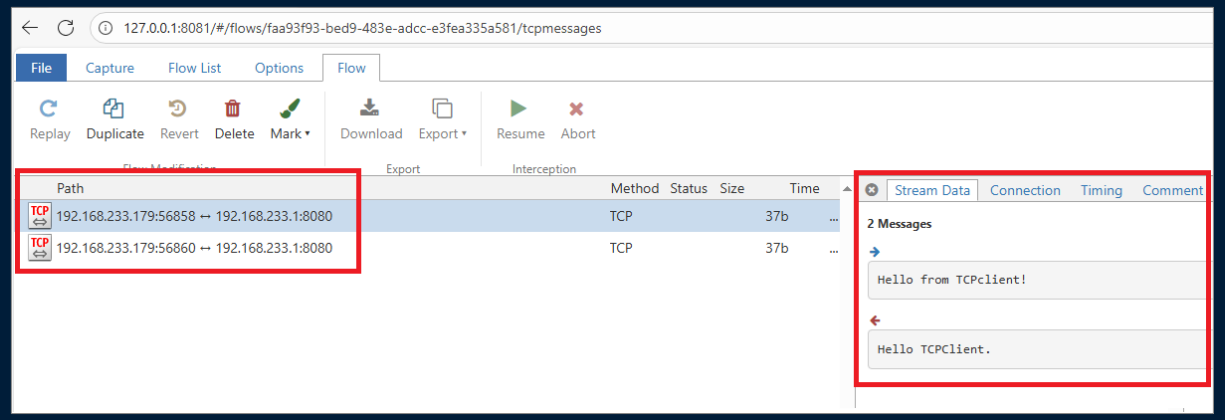
- We can use Wireshark to check the protocol. As it is unencrypted, we can even read the content



Scenario 3 – Lab Example 1

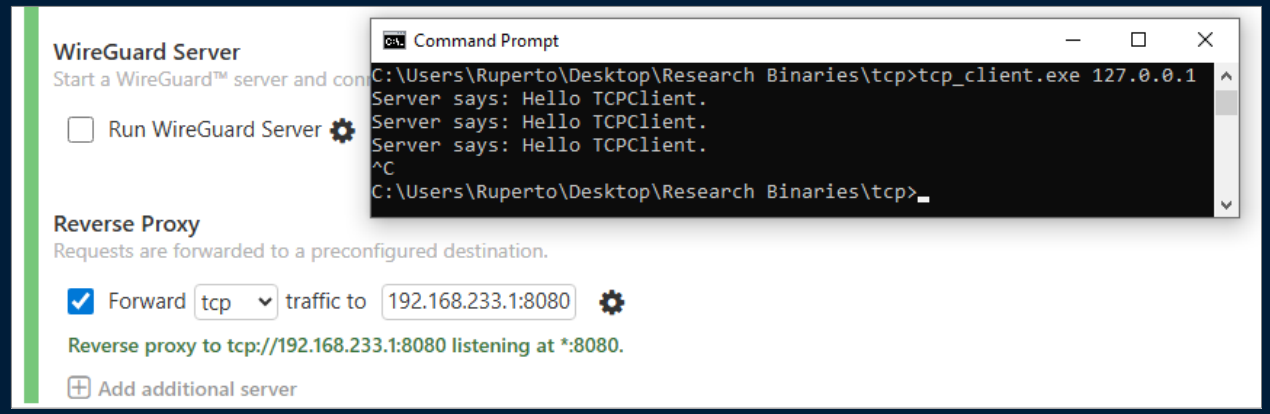
- Intercepting the traffic is easy. However, Burp is not the best option
- MITM Proxy is a better option

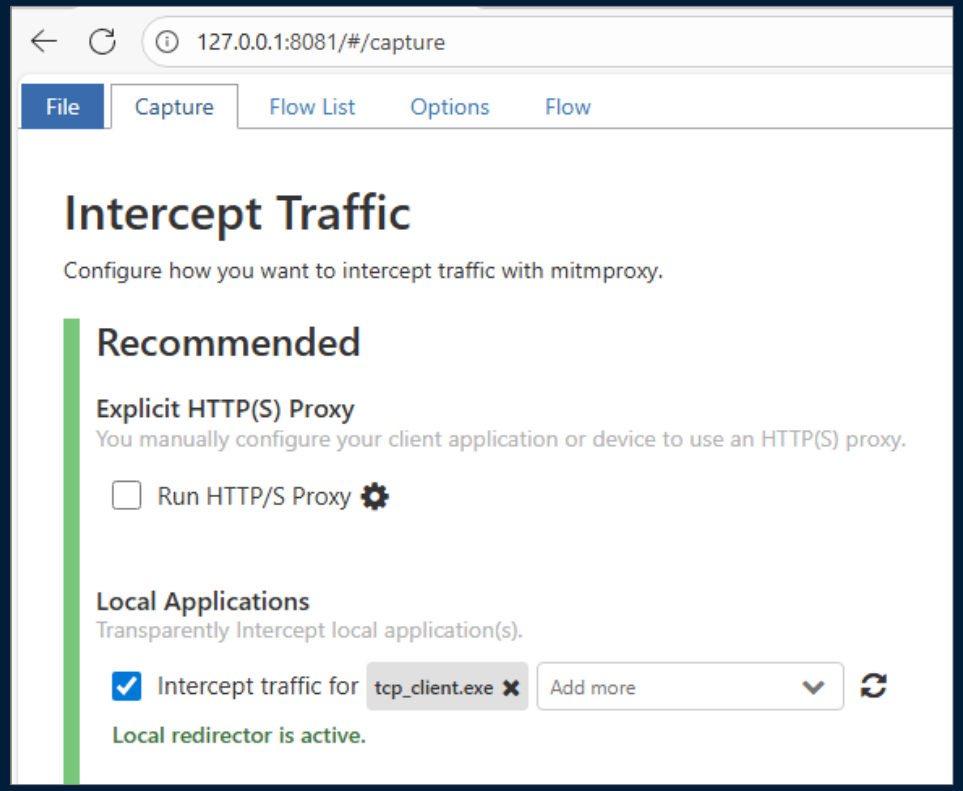




Scenario 3 – Lab Example 1

- For MITM Proxy we can intercept the traffic in two ways:
  - Setting a local forwarder (Local Proxy listener)
  - Hooking the process





Scenario 4 – Intercepting TLS over TCP

- Intercepting TLS starts to be challenging
  - Burp is no longer an option.
  - Certificate/Hostname Validation and Certificate Pinning are an extra layer to avoid.
  - Lack of tools for “plug and play”
  - It might require custom scripts
  - Sometimes, it almost impossible to intercept*

- Situations we might face**:
  - Case 1: The agent performs Certificate Validation – Can be bypassed by adding the proxy certificate to our vault or even using the CA certificate as the proxy’s.
  - Case 2: The agent performs some sort of Server Validation (Certificate Pinning or Hostname Validation)
    - Can be bypasses it If the CA is under our control.
    - Pinning can be disabled patching the binary

*Sometimes, changing the hardcoded certificate via binary patching is enough too

** assuming we can set a proxy as before

Scenario 4 – Lab Example 1

- ***Tls_agent.exe*** connects to ***tls_server.exe*** using a hardcoded hostname. The agent performs Certificate Validation.

```
C:\Users\Ruperto\Desktop\Research Binaries\tls over tcp>tls_agent.exe
Certificate validated successfully by Windows
Message sent to server: Hello, I'm the TLS agent!
Reply from server: Hello agent, I'm your server!
```

```
C:\Users\Jorge\Downloads\TEST>tls_server.exe
TLS server listening on port 8443...
Received from agent: Hello, I'm the TLS agent!
```

Scenario 4 – Lab Example 1

- As before, we can point it to MITM Proxy by changing the IP in the /etc/hosts file

hosts | tcp_client.cpp | tcp_server.cpp | client_tls.c | server_tls.c

```
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host nam
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#      102.54.94.97     rhino.acme.com          # source server
#       38.25.63.10     x.acme.com              # x client host

# localhost name resolution is handled within DNS itself.
#   127.0.0.1       localhost
#   ::1             localhost
192.168.152.100 kali
127.0.0.1 decrypting.lab
```

Scenario 4 – Lab Example 1

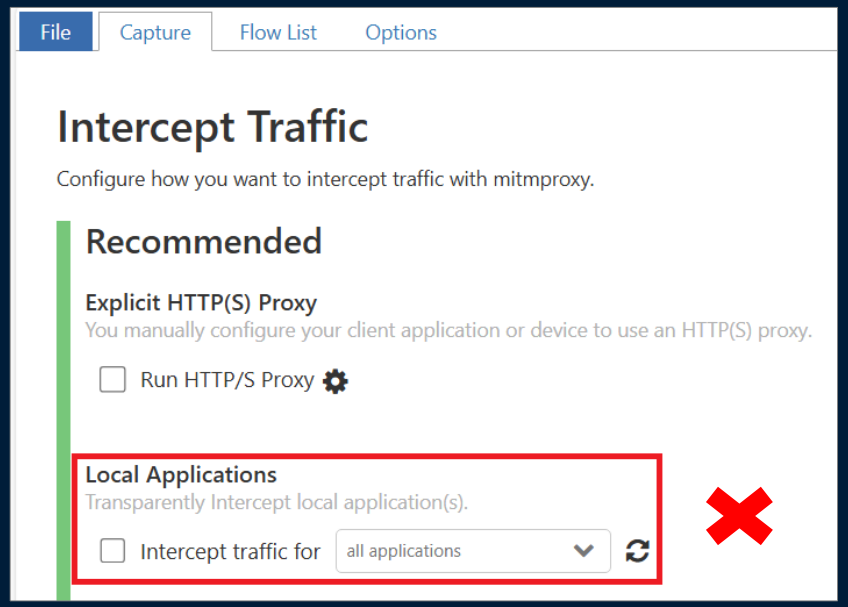
- However, as the certificate validation is in place, intercepting won't work by default.

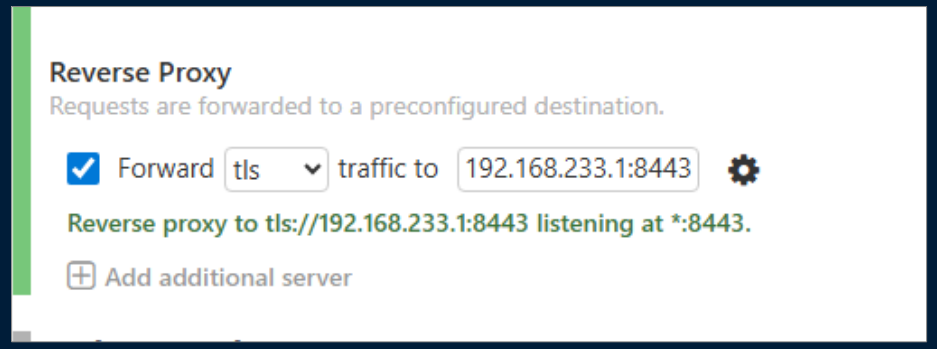
```
C:\Users\Ruperto\Desktop\Research Binaries\tls over tcp>tls_agent.exe
Certificate is NOT trusted. TrustStatus: 0x00010000
Connection aborted: certificate is not trusted.
```

```
[03:27:29.990][192.168.233.179:57422] server connect 192.168.233.1:8443
[03:27:30.023][192.168.233.179:57422] Server TLS handshake failed. Certificate verify failed: unable to get local issuer
 certificate
[03:27:30.030][192.168.233.179:57422] Unable to establish TLS connection with server (Certificate verify failed: unable
to get local issuer certificate). Trying to establish TLS with client anyway. If you plan to redirect requests away from
 this server, consider setting `connection_strategy` to `lazy` to suppress early connections.
[03:27:30.053][192.168.233.179:57422] server disconnect 192.168.233.1:8443
[03:27:30.278][192.168.233.179:57422] client disconnect
```

Quick Parenthesis – MITM Proxy

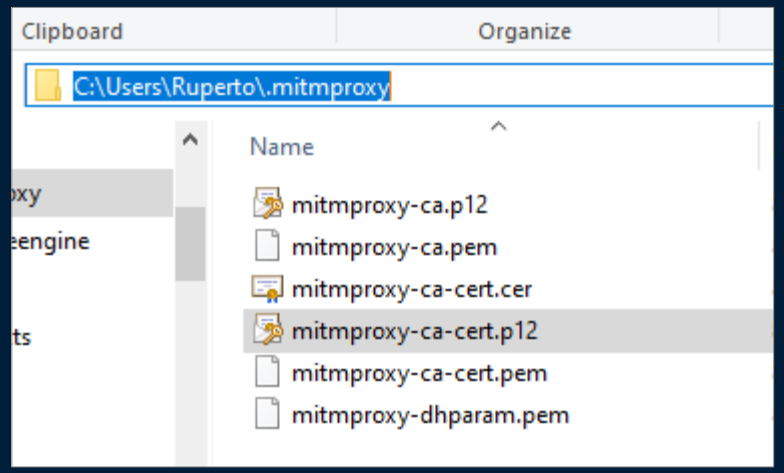
- Hooking the application with MITM Proxy does not work for bypassing the certificate validation issue. MITM Proxy expect HTTPs. So we need to use the Reverse Proxy functionality to intercept TLS over TCP.

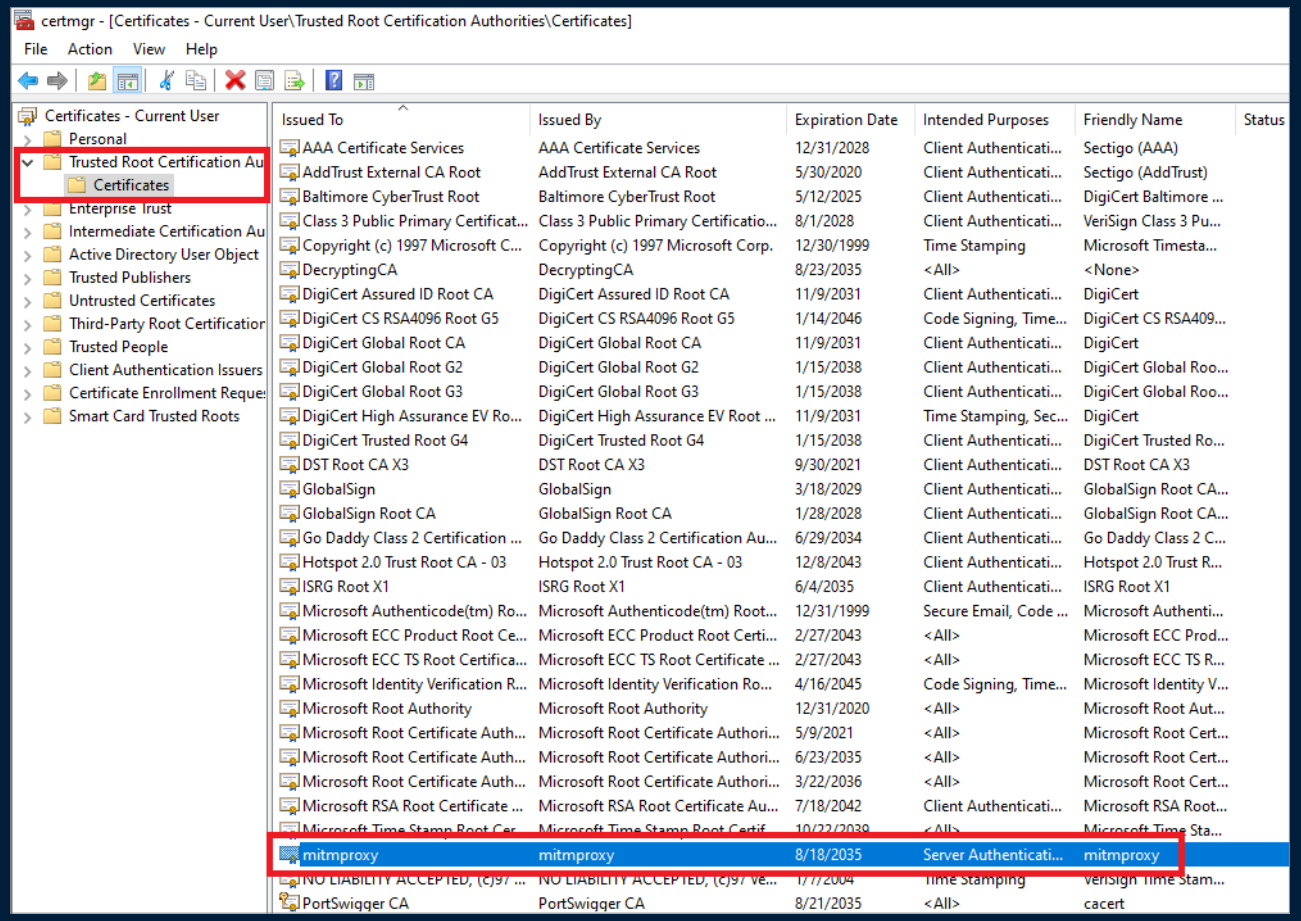




Scenario 4 – Lab Example 1

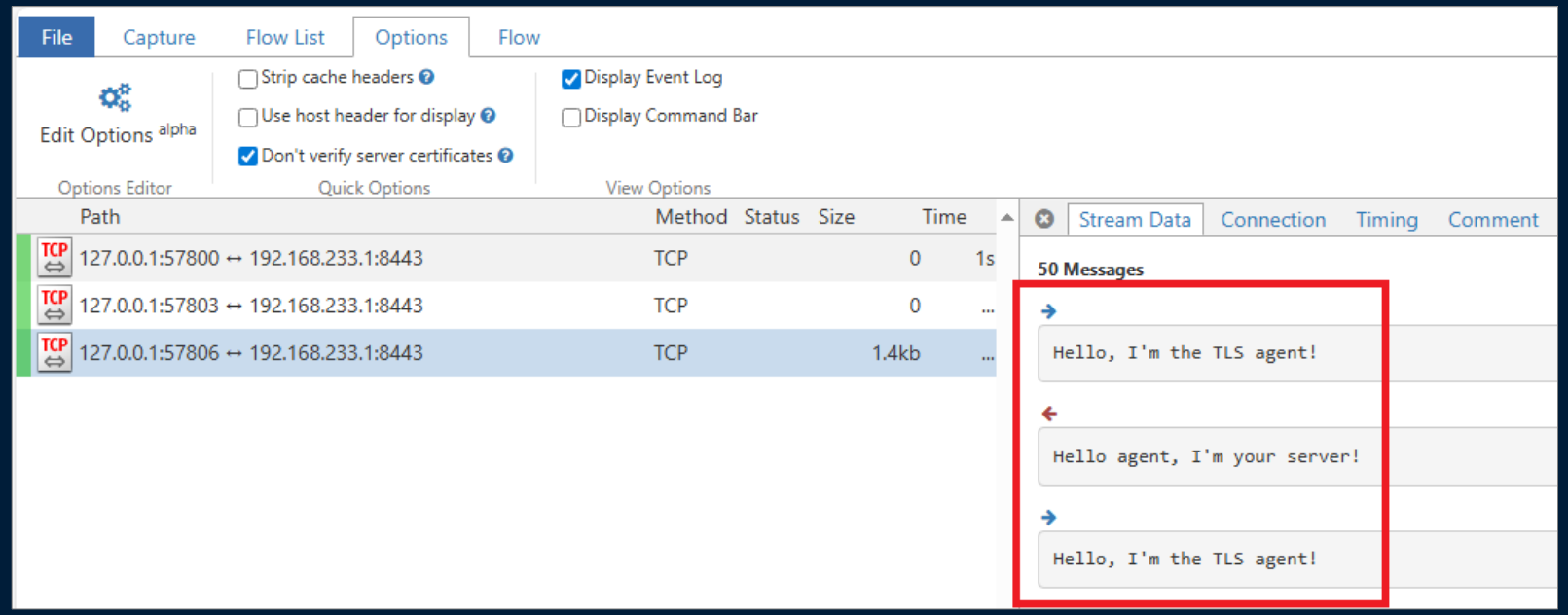
- For fixing the certificate issue, we can install the certificate in the Trusted Root Certification Authority Vault.





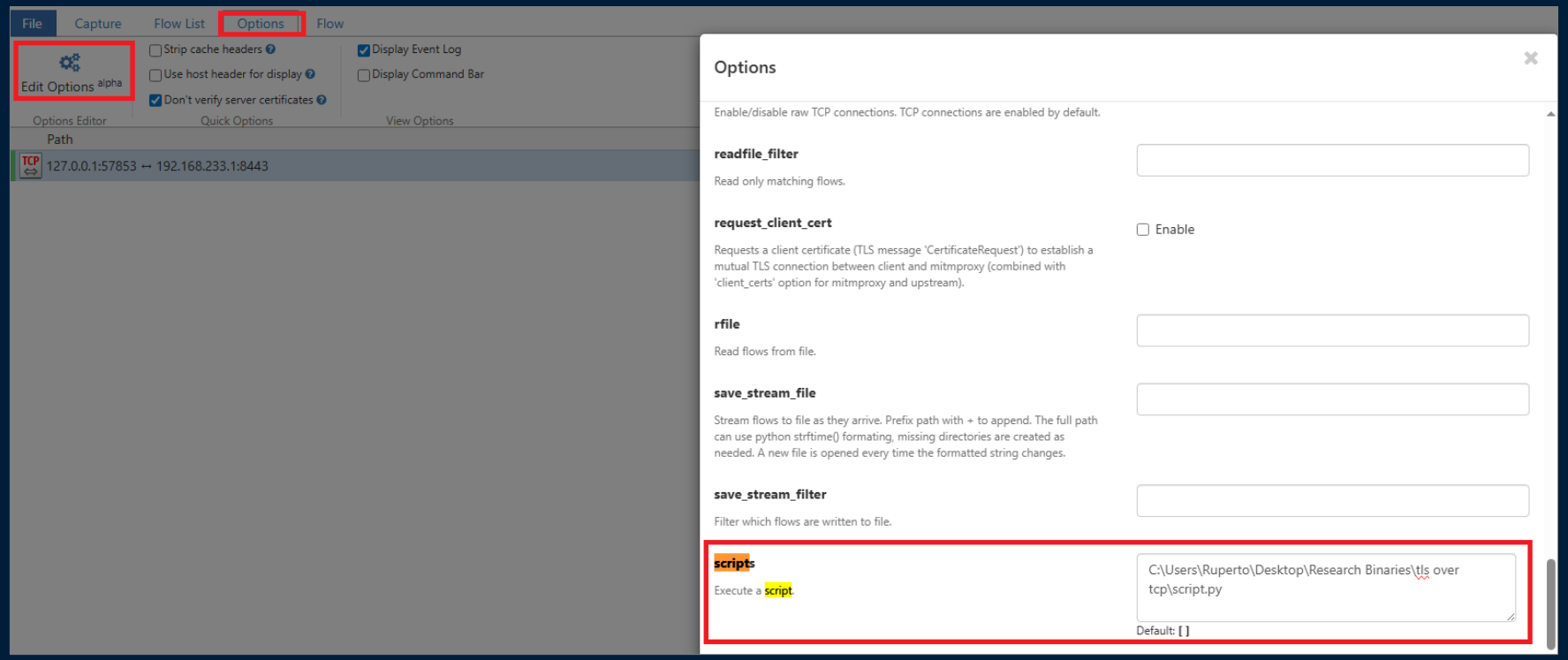
Scenario 4 – Lab Example 1

- After that, we should be able to intercept the traffic.



Scenario 4 – Lab Example 1

- Moreover, we can even modify the content on the fly using the parameter scripts.



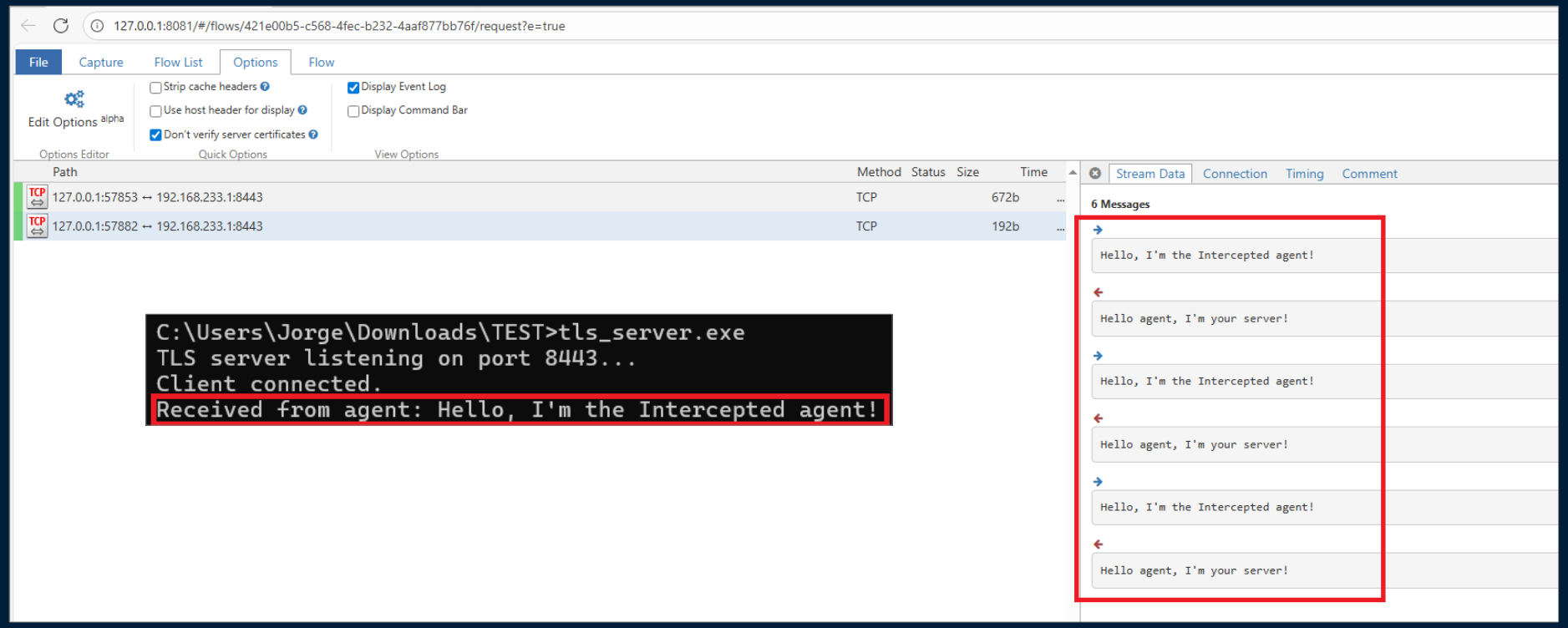
Scenario 4 – Lab Example 1

- For example, we can modify the content of the intercepted message using the Built-In addons.

```python
"""
Process individual messages from a TCP connection.

This script replaces full occurrences of "foo" with "bar" and prints various details for each message.
Please note that TCP is stream-based and *not* message-based. mitmproxy splits stream contents into "messages"
as they are received by socket.recv(). This is pretty arbitrary and should not be relied on.
However, it is sometimes good enough as a quick hack.

Example Invocation:

    mitmdump --tcp-hosts ".*" -s examples/tcp-simple.py
"""

import logging

from mitmproxy import tcp
from mitmproxy.utils import strutils


def tcp_message(flow: tcp.TCPFlow):
    message = flow.messages[-1]
    message.content = message.content.replace(b"TLS agent", b"Intercepted agent")

    logging.info(
        f"tcp_message[from_client={message.from_client}), content={strutils.bytes_to_escaped_str(message.content)}]"
    )
```

Scenario 4 – Lab Example 1

- Here is the result:



Scenario 4 – Lab Example 2

- ***Client.exe*** connects to ***Server.exe*** using a hardcoded hostname. Both of them are Golang binaries.
- Golang applications that use HTTPs/TLS (crypto/tls) requests have a built-in SSL verification feature enabled by default (InsecureSkipVerify).

```
C:\Users\Jorge\Downloads\TEST\golang>server.exe
TLS server listening on port 443...
Received from client: Hello server, I´m a Golang agent!
```

```
C:\Users\Ruperto\Desktop\Research Binaries\go>client.exe
Server replied: Hello golang agent!
Message sent successfully: Hello server, I´m a Golang agent!
```

Scenario 4 – Lab Example 2

- As before, we can point it to MITM Proxy by changing the IP in the /etc/hosts file. MITM Proxy certificate is also installed.
- However, due to the built-in validation, this trick does not work.

```
C:\Users\Ruperto\Desktop\Research Binaries\go>client.exe
2025/09/05 01:48:33 TLS connection failed: tls: failed to verify certif
icate: x509: certificate signed by unknown authority
```

hosts | tcp_client.cpp | tcp_server.cpp | client_tls.c | server_tls.c

```
1  # Copyright (c) 1993-2009 Microsoft Corp.
2  #
3  # This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
4  #
5  # This file contains the mappings of IP addresses to host names. Each
6  # entry should be kept on an individual line. The IP address should
7  # be placed in the first column followed by the corresponding host nam
8  # The IP address and the host name should be separated by at least one
9  # space.
10 #
11 # Additionally, comments (such as these) may be inserted on individual
12 # lines or following the machine name denoted by a '#' symbol.
13 #
14 # For example:
                                          rhino.acme.com          # source server
                                          x.acme.com              # x client host
                                          tion is handled within DNS itself.
20 #   127.0.0.1       localhost
21 #   ::1             localhost
22 192.168.152.100 kali
23 127.0.0.1 decrypting.lab
```

Scenario 4 – Lab Example 2

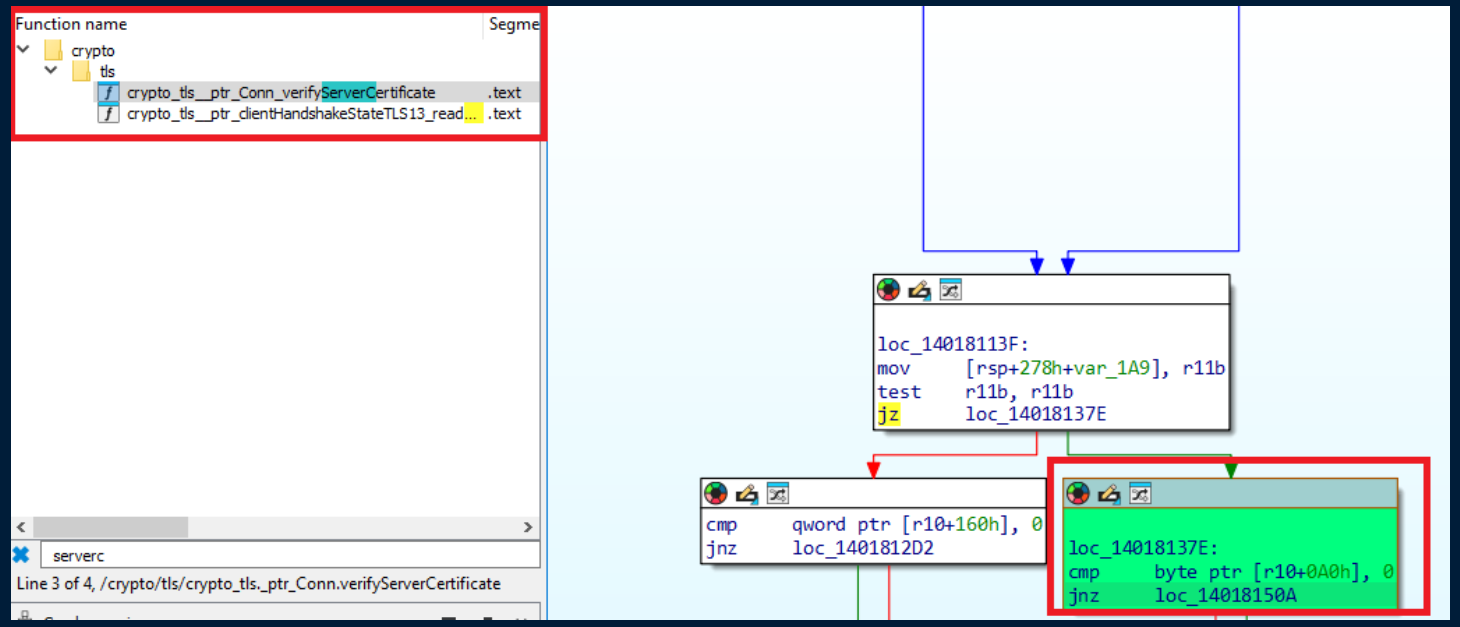
- The solution proposed by Cyberark is patching the binary to avoid the SSL verification.
- TL;DR -> InsecureSkipVerify will be always “set” to True.
- If set to False, the validation will be performed.

```
// InsecureSkipVerify controls whether a client verifies the server's
// certificate chain and host name. If InsecureSkipVerify is true, crypto/tls
// accepts any certificate presented by the server and any host name in that
// certificate. In this mode, TLS is susceptible to machine-in-the-middle
// attacks unless custom verification is used. This should be used only for
// testing or in combination with VerifyConnection or VerifyPeerCertificate.
InsecureSkipVerify bool
```

```
// TLS configuration with proper certificate validation
tlsConfig := &tls.Config{
    RootCAs:    caPool,
    ServerName: "decrypting.lab", // Must match the server cert's CN or SAN
}
```

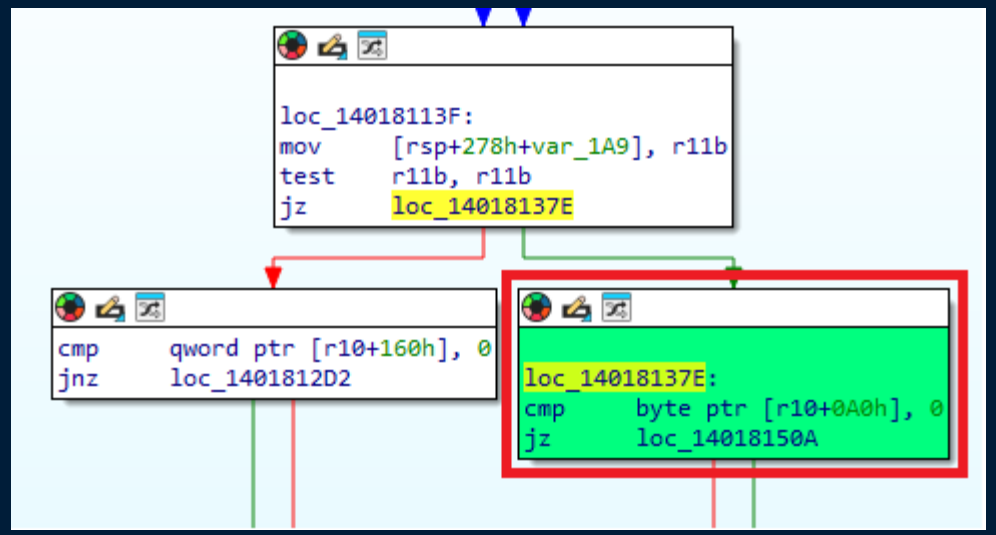
Scenario 4 – Lab Example 2

- If the comparison is True, then Zero Flag is set to 0 => Certificate provided is valid => The jump is not taken (SSL Verification Bypass).
- If the comparison is False, then Zero Flag is cleared (not zero) => The jump is taken (SSL Verification).



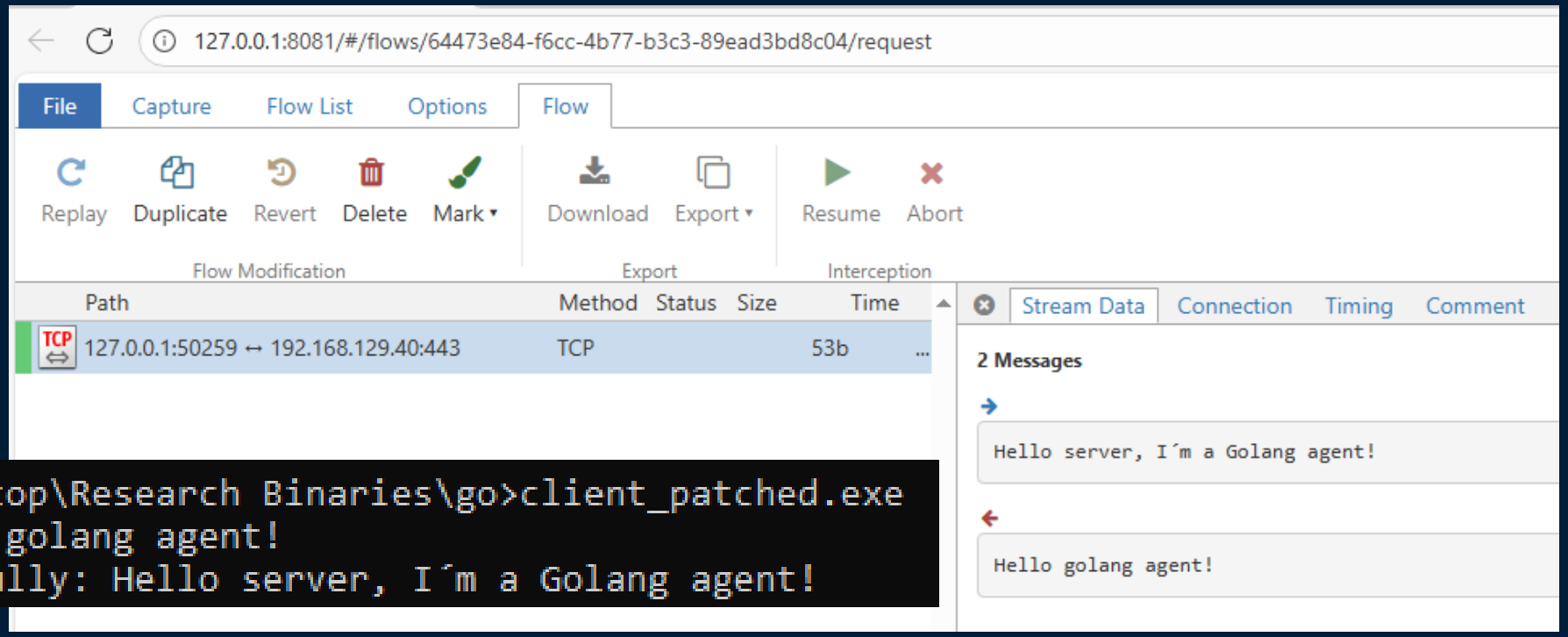
Scenario 4 – Lab Example 2

- So, as we are providing an invalid certificate, we want to make sure that the jump is not taken.
- To do that, we can change the JUMP instruction. As Zero Flag is going to be always different to 0, by changing JNZ to JZ, will make that the JUMP is taken only when Zero Flag is Zero. In other words, the “validation” is bypassed and the execution can continue.



Scenario 4 – Lab Example 2

- Now intercepting is possible again.




```
C:\Users\Ruperto\Desktop\Research Binaries\go>client_patched.exe
Server replied: Hello golang agent!
Message sent successfully: Hello server, I´m a Golang agent!
```

Scenario 5 – Intercepting mutual TLS

- Intercepting mutual TLS is the worst scenario we can face.
- If we don’t have access to the CA, intercepting the traffic might be impossible*.
- Certificate Validation or Certificate Pinning can be also added on top of it.

- Situations we might face:
  - Case 1: We have access to the CA.
  - Case 2: We don’t.

*Nothing is impossible, just harder

Scenario 5 – Lab Example 1

- ***Mtls_client.exe*** connects to ***mtls_serrver.exe*** using a hardcoded hostname.

```
C:\Users\Jorge\Downloads\TEST>mtls_server.exe
TLS Server (Mutual TLS) listening on port 8443...
Client authenticated: /CN=decrypting.client.lab
Received: mTLS agent saying Hi!
```

```
C:\Users\Ruperto\Desktop\Research Binaries\mutual tls>mtls_client.exe
Server authenticated: /CN=decrypting.lab
Server says: Hello mTLS agent. I'm your server!
```

Scenario 5 – Lab Example 1

- Same step as before: change the /etc/host file to point to MITM Proxy
- This time, installing MITM Proxy's certificate in the Trusted Root Store won't change anything as the agent is expecting the server's certificate.

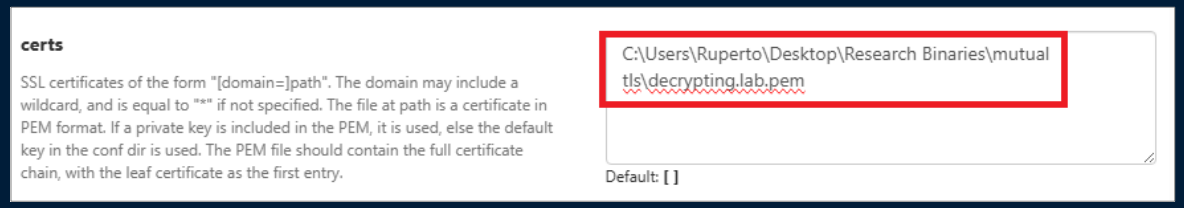
```
[05:16:49.047][127.0.0.1:57929] server connect 192.168.233.1:8443
[05:16:49.138][127.0.0.1:57929] TLS Error: [('SSL routines', '', 'tlsv13 alert certificate required')]
[05:16:49.148][127.0.0.1:57929] Client TLS handshake failed. The client does not trust the proxy's certificate for decry
pting.lab (tlsv1 alert unknown ca)
[05:16:49.166][127.0.0.1:57929] server disconnect 192.168.233.1:8443
[05:16:49.204][127.0.0.1:57929] client disconnect
```

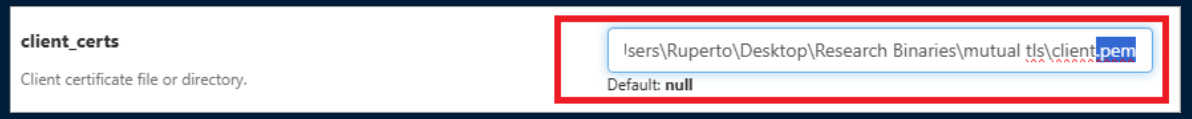
hosts | mtls_client.c | mtls_server.c

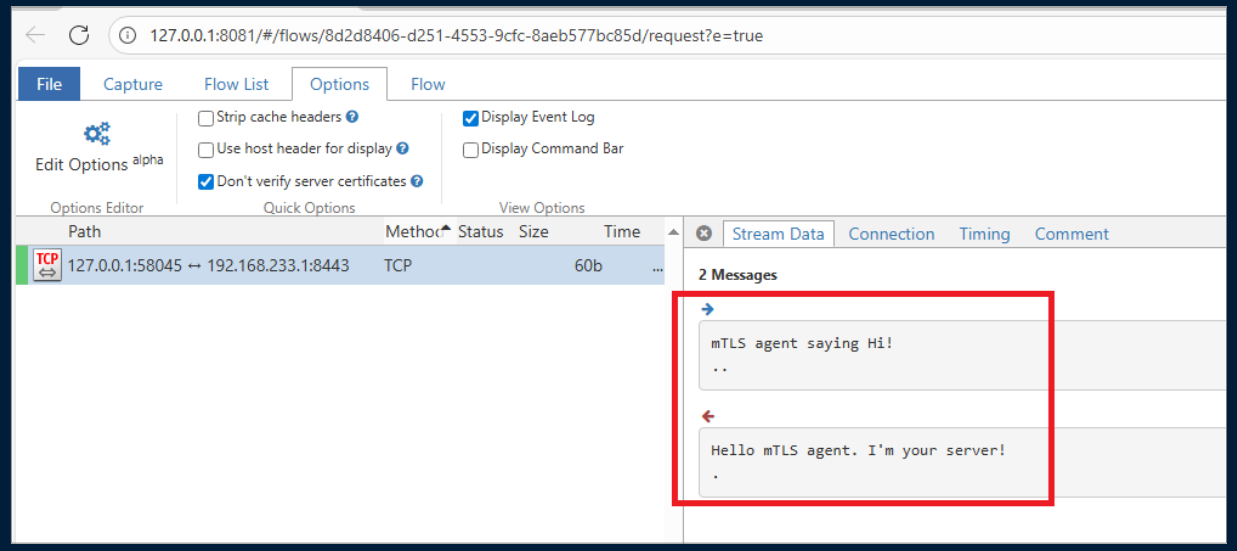
```
1  # Copyright (c) 1993-2009 Microsoft Corp.
2  #
3  # This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
4  #
5  # This file contains the mappings of IP addresses to host names. Each
6  # entry should be kept on an individual line. The IP address should
7  # be placed in the first column followed by the corresponding host name.
8  # The IP address and the host name should be separated by at least one
9  # space.
10 #
11 # Additionally, comments (such as these) may be inserted on individual
12 # lines or following the machine name denoted by a '#' symbol.
13 #
14 # For example:
                                  rhino.acme.com          # source server
                                  x.acme.com              # x client host
                                  ution is handled within DNS itself.
20 #   127.0.0.1       localhost
21 #   ::1             localhost
22 192.168.152.100 kali
23 127.0.0.1 decrypting.lab
```

Scenario 5 – Lab Example 1

- As we have access to the CA, we can avoid the validation and intercept the traffic using MITM Proxy

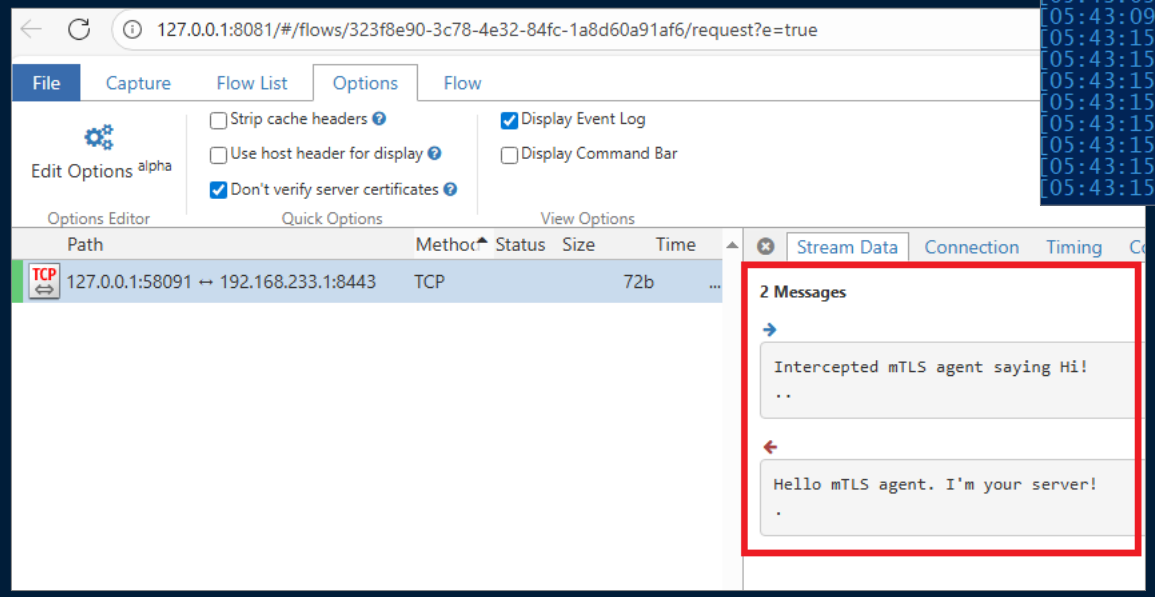






Scenario 5 – Lab Example 1

- Now, we can modify the content again as before




```
[05:41:19.110][127.0.0.1:58083] server disconnect 192.168.233.1:8443
[05:41:19.117][127.0.0.1:58083] client disconnect
[05:43:09.151] Un-loading script: C:\Users\Ruperto\Desktop\Research Binaries\mutual tls\script.py
[05:43:09.162] Loading script C:\Users\Ruperto\Desktop\Research Binaries\mutual tls\script2.py
[05:43:15.353][127.0.0.1:58091] client connect
[05:43:15.362][127.0.0.1:58091] server connect 192.168.233.1:8443
[05:43:15.450] === Modified Agent Request ===
[05:43:15.453] Original: mTLS agent saying Hi!\n\x00\x00
[05:43:15.455] Modified: Intercepted mTLS agent saying Hi!\n\x00\x00
[05:43:15.474] ==============================
[05:43:15.607][127.0.0.1:58091] server disconnect 192.168.233.1:8443
[05:43:15.627][127.0.0.1:58091] client disconnect
```

Diving Deeper: Intercepting with Frida

- What can we do when the CA is not part of the equation?
  - MITM is no longer usable.
  - Need to dive deeper -> Frida might be an option (or any debugger)
- Frida allows you to:
  - inject your own scripts into black box processes.
  - Hook any function, spy on crypto APIs or trace private application code, no source code needed.
  - Edit, hit save, and instantly see the results. All without compilation steps or program restarts.
  - Drawbacks – Not user friendly.

FRIDA

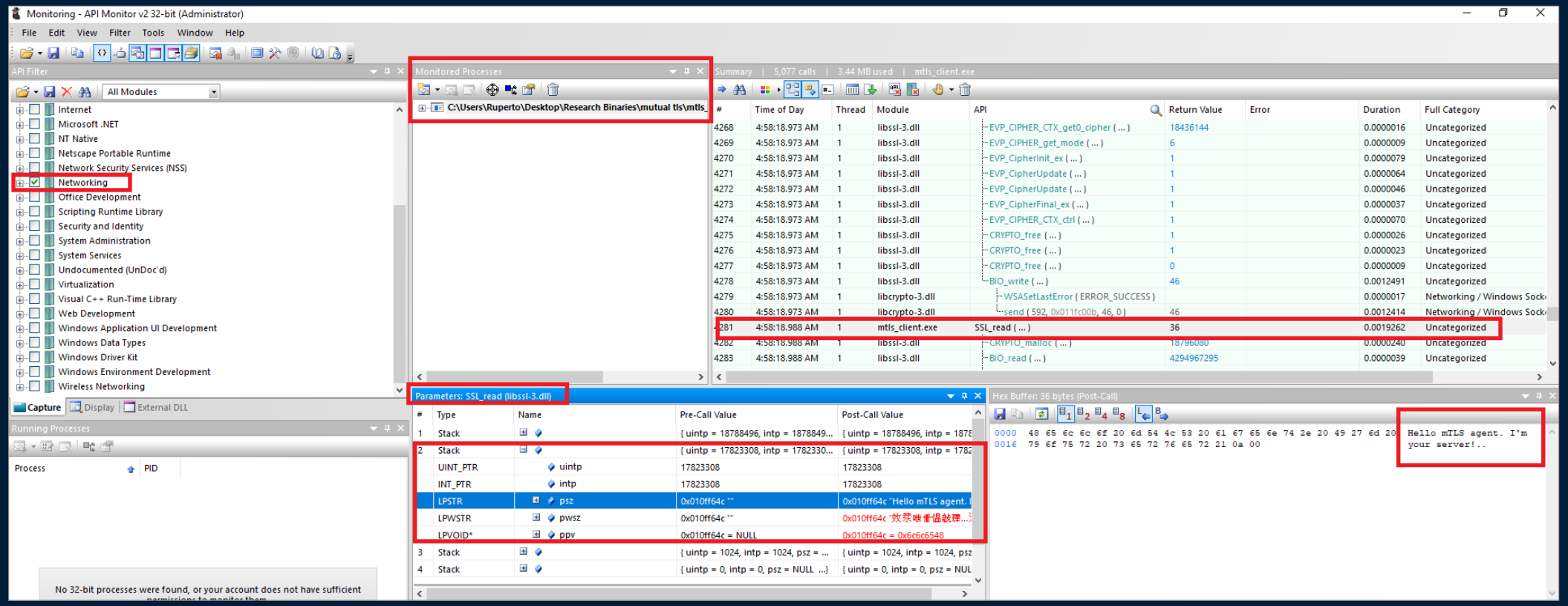
Diving Deeper: Intercepting with Frida

- In May 2025, Frida 17 was released. Several changes were implemented (API names are no longer the same). This means, most of the public resources related to Frida are now outdated.
- However, StackOverflow is always there for us.
- Before: Module.getGlobalExportByName(null, 'open'); → After: Module.getGlobalExportByName('open');

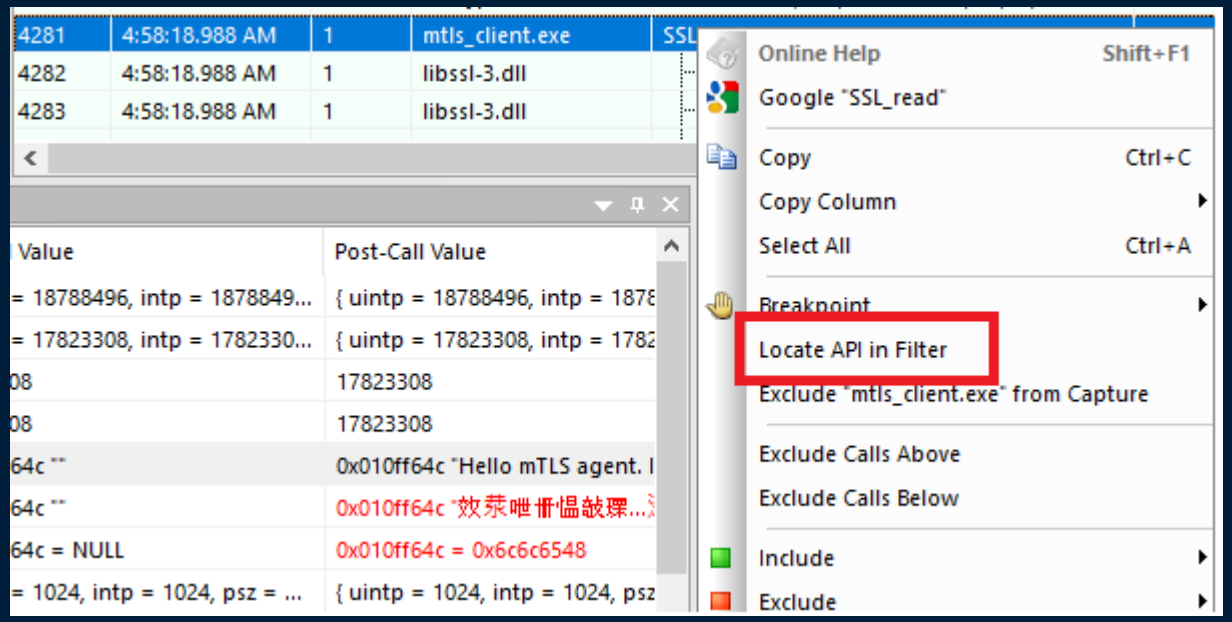
Diving Deeper: Intercepting with Frida

- Let's face *Scenario 5 – Example 1* using Frida but, this time, without having access to the CA certificate.
- To do so, we have to:
  - Identify the API used to send the payload.
  - Hook it.
  - Modify the content.
  - Profit!
- Depending on the language of the application, identifying the API calls can be extremely challenging (Golang) or straightforward (C, C+, C++).
- API Monitor can be an excellent complement for this labor.

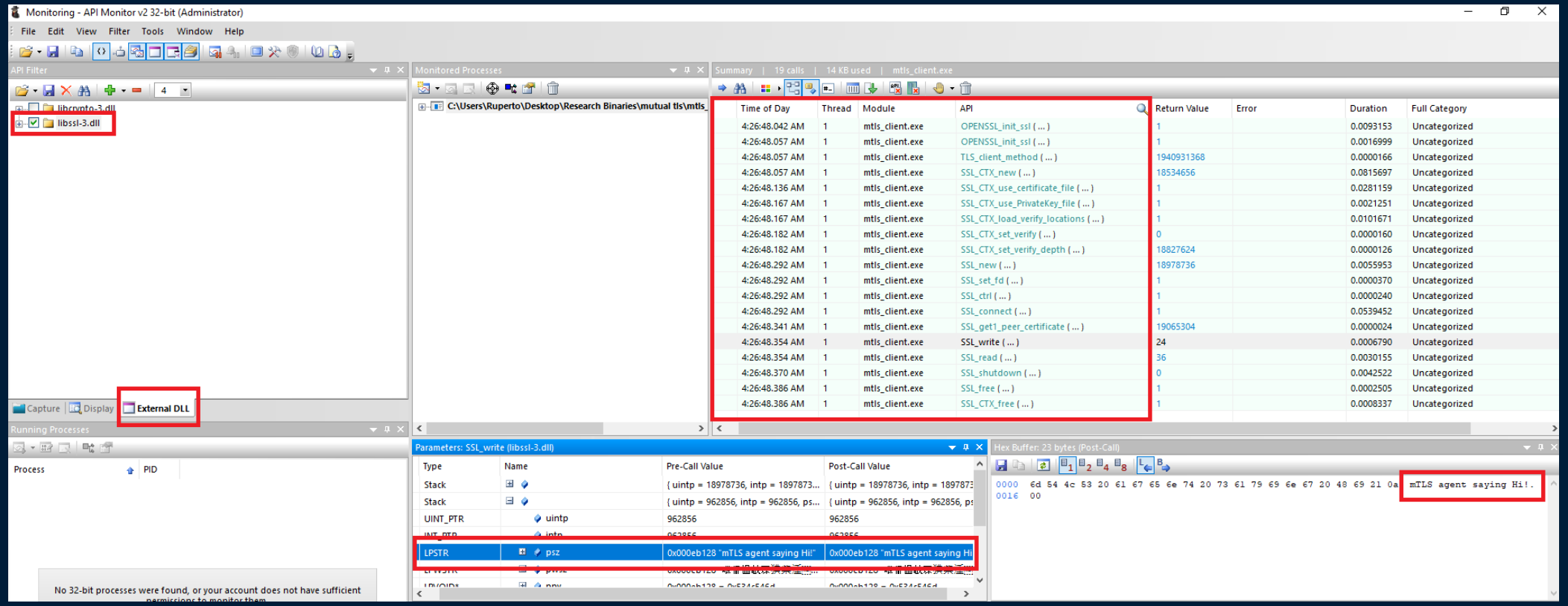
Diving Deeper: Intercepting with Frida



Diving Deeper: Intercepting with Frida



Diving Deeper: Intercepting with Frida



Diving Deeper: Intercepting with Frida

- Using Frida, we can hook SSL_Read() and SSL_Write() API calls to see the messages exchanged between the agent and the server.

```
'use strict';

const ssl_read_ptr = Process.getModuleByName("libssl-3.dll").getExportByName("SSL_read");
const ssl_write_ptr = Process.getModuleByName("libssl-3.dll").getExportByName("SSL_write");

Interceptor.attach(ssl_read_ptr, {
  onEnter: function (args) {
    this.buf = args[1];
  },
  onLeave: function (retval) {
    let len = -1;
    try { len = retval.toInt32(); } catch (e) { return; }
    if (len > 0 && this.buf && !this.buf.isNull()) {
      try {
        const data = this.buf.readByteArray(len);
        if (data) send({ type: 'ssl_read', len: len }, data);
      } catch (e) {}
    }
  }
});

Interceptor.attach(ssl_write_ptr, {
  onEnter: function (args) {
    this.buf = args[1];
    let len = -1;
    try { len = args[2].toInt32(); } catch (e) { return; }
    if (len > 0 && this.buf && !this.buf.isNull()) {
      try {
        const data = this.buf.readByteArray(len);
        if (data) send({ type: 'ssl_write', len: len }, data);
      } catch (e) {}
    }
  }
});
```

Diving Deeper: Intercepting with Frida

```
C:\Users\Ruperto\Desktop\Research Binaries\frida>frida -f mtls_client.exe -l intercept.js
     ____
    / _  |   Frida 17.2.12 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Local System (id=local)
Spawned `mtls_client.exe`. Resuming main thread!
[Local::mtls_client.exe ]-> message: {'type': 'send', 'payload': {'type': 'ssl_write', 'len': 30}} data: b'mTLS agent saying Hi!\x00\x00\x00\x00\x00\x00\x00\x00\x00'
message: {'type': 'send', 'payload': {'type': 'ssl_read', 'len': 36}} data: b"Hello mTLS agent. I'm your server!\n\x00"
Process terminated
[Local::mtls_client.exe ]->

Thank you for using Frida!
```

Diving Deeper: Intercepting with Frida

- We can do the same to alter the traffic once we know the information sent.

```
Interceptor.attach(ssl_read_ptr, {
  onEnter: function (args) {
    this.buf = args[1];
  },
  onLeave: function (retval) {
    let len = -1;
    try { len = retval.toInt32(); } catch (e) { return; }
    if (len > 0 && this.buf && !this.buf.isNull()) {
      try {
        const data = this.buf.readByteArray(len);
        if (data) send({ type: 'ssl_read', len: len }, data);
      } catch (e) {}
    }
  }
});
```

```
Interceptor.attach(ssl_write_ptr, {
  onEnter(args) {
    this.buf = args[1];
    this.len = -1;
    try { this.len = args[2].toInt32(); } catch (e) { this.len = -1; }

    if (this.len <= 0 || !this.buf || this.buf.isNull()) return;

    // --- create an immutable copy of the original bytes ---
    try {
      const origAB = this.buf.readByteArray(this.len);          // ArrayBuffer
      const origCopy = origAB ? Uint8Array.from(new Uint8Array(origAB)).buffer : null; // real copy
      if (origCopy) {
        send();
        send({ type: 'ssl_write', len: this.len, stage: 'original' }, origCopy);
      }
    } catch (e) {
      send({ type: 'ssl_write_err_read_original', message: e.message });
    }

    // --- do in-place replacement BEFORE the real SSL_write runs ---
    try {
      const count = replaceInBuffer(this.buf, this.len, findBytes, replBytes);
      if (count > 0) {
        try {
          const modAB = this.buf.readByteArray(this.len);
          const modCopy = modAB ? Uint8Array.from(new Uint8Array(modAB)).buffer : null;
          if (modCopy) {
            //send({ type: 'ssl_write', len: this.len, stage: 'String modified', count: count, find: FIND, repl: REPL });
            send({ type: 'ssl_write', len: this.len, stage: 'modified' },modCopy);
          }
        } catch (e) {
```

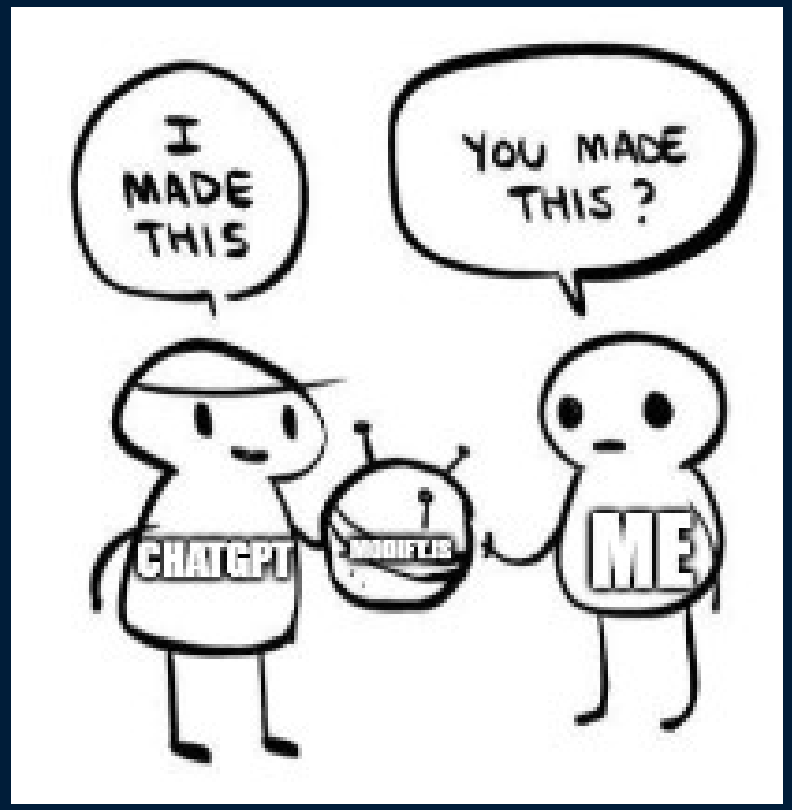
Diving Deeper: Intercepting with Frida

```
C:\Users\Ruperto\Desktop\Research Binaries\frida>frida -f mtls_client.exe -l modify.js
     ____
    / _  |   Frida 17.2.12 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Local System (id=local)
Spawned `mtls_client.exe`. Resuming main thread!
[Local::mtls_client.exe ]-> message: {'type': 'send'} data: None
message: {'type': 'send', 'payload': {'type': 'ssl_write', 'len': 30, 'stage': 'original'}} data: b'mTLS agent saying Hi!\x00\x00\x00\x00\x00\x00\x00\x00\x00'
message: {'type': 'send', 'payload': {'type': 'ssl_write', 'len': 30, 'stage': 'modified'}} data: b'mTLS agent is pwned!!\x00\x00\x00\x00\x00\x00\x00\x00\x00'
message: {'type': 'send', 'payload': {'type': 'ssl_read', 'len': 36}} data: b"Hello mTLS agent. I'm your server!\n\x00"
Process terminated
[Local::mtls_client.exe ]->

Thank you for using Frida!
```

Windows PowerShell

```
PS C:\Users\Jorge\Downloads\TEST> .\mtls_server.exe
TLS Server (Mutual TLS) listening on port 8443...
Client authenticated: /CN=decrypting.client.lab
Received: mTLS agent is pwned!!
```

Diving Deeper: Intercepting with Frida



Decision Tree Diagram

Real Word Example – Sliver C2

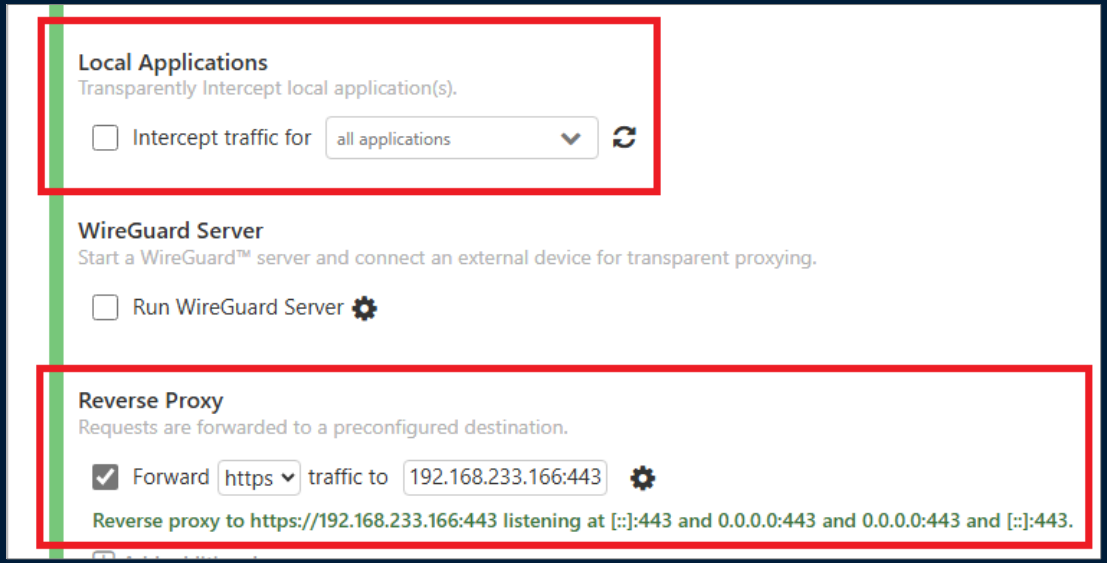
- Sliver is an open C2 framework. Sliver's implants support C2 over Mutual TLS (mTLS), WireGuard, HTTP(S), and DNS and are dynamically compiled with per-binary asymmetric encryption keys. It’s primary language is Golang.
- Why Sliver? It’s open source.
- Goal: Intercept HTTPs* and mTLS traffic

HTTPS C2

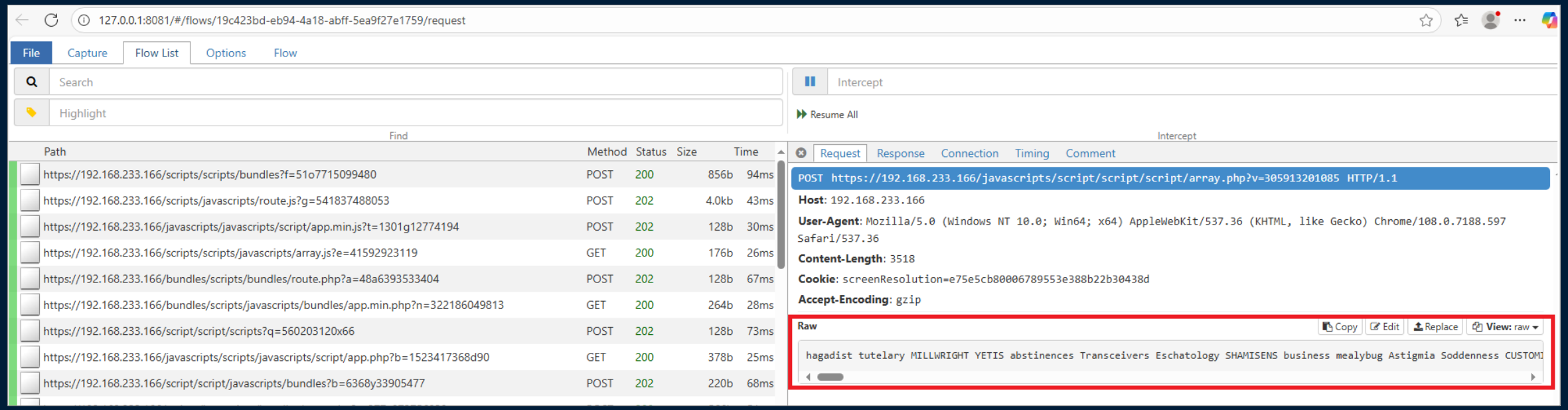
Sliver supports proxy-aware C2 over both HTTP and HTTPS, however since Sliver does not rely upon the SSL/TLS layer for security these protocols are considered somewhat synonymous. There are separate commands for the listeners but an implant generated with `--http` may attempt to connect over both HTTP and HTTPS (see "under the hood" for more details).

Sliver C2 – Intercepting HTTPs

- Beacon generation: *generate beacon --http decrypting.lab:443 --os windows --arch amd64 --format exe --save* https_not_obfuscated.exe --disable-sgn --skip-symbols
- Sliver configuration: *https -L decrypting.lab -l 443*
- /etc/hosts should point to Sliver's IP
- We can use MITM Proxy to intercept the traffic. Burp Suite should work too.
- MITM Proxy command line: *.\mitmweb.exe --mode reverse:https://<Sliver IP>:443 --listen-port 443 --ssl-insecure*



Sliver C2 – Intercepting HTTPs



- Traffic can be fully de-obfuscated using [SliverC2-Forensisc]

Sliver C2 – Intercepting mTLS

- For decrypting mTLS we need the server's and the client's certificate. Not a real life approach.
- Beacon generation: *generate beacon --mtls decrypting.lab --os windows --arch amd64 --format exe --save mtlsbeacon_not_obfuscated.exe --disable-sgn --skip-symbols*
- Sliver configuration: *mtls -L decrypting.lab -l 8888*
- /etc/hosts should point to Sliver's IP
- We can use MITM Proxy to intercept the traffic
- MITM Proxy command line: *.\mitmweb.exe --mode reverse:tls://<Sliver IP>:8888 --listen-port 8888 --certs="mtls-server-ca-cert.pem" --set client_certs="mtls-implant-ca-cert.pem" --ssl-insecure --script "script.py"*

```
pired
[02:05:00.149][127.0.0.1:50367] client disconnect
[02:05:17.578][127.0.0.1:50370] client connect
[02:05:17.586][127.0.0.1:50370] server connect 192.168.233.166:8888
[02:05:17.693] Addon error: Attribute's length must be >= 1 and <= 64, but it was 0
Traceback (most recent call last):
  File "mitmproxy\addons\tlsconfig.py", line 221, in tls_start_client
  File "mitmproxy\addons\tlsconfig.py", line 635, in get_cert
  File "mitmproxy\certs.py", line 693, in get_cert
  File "mitmproxy\certs.py", line 345, in dummy_cert
  File "cryptography\x509\name.py", line 152, in __init__
ValueError: Attribute's length must be >= 1 and <= 64, but it was 0
[02:05:17.712][127.0.0.1:50370] No TLS context was provided, failing connection.
[02:05:17.717][127.0.0.1:50370] client disconnect
```

Sliver C2 – Intercepting mTLS

```
pentester@kali ~/.sliver % cd certs
pentester@kali ~/.sliver/certs % ls
https-ca-cert.pem  https-ca-key.pem  mtls-implant-ca-cert.pem  mtls-implant-ca-key.pem  mtls-server-ca-cert.pem  mtls-server-ca-key.pem  operator-ca-cert.pem  operator-ca-key.pem
pentester@kali ~/.sliver/certs % cat mtls-server-ca-cert.pem
-----BEGIN CERTIFICATE-----
MIIB+DCCAVmgAwIBAgIRANLSAiRfUiAo9oYpufJbB2YwCgYIKoZIzj0EAwQwADAe
Fw0yNTAyMTMxNDQ3NTJaFw0yNzAyMTMxNDQ3NTJaMAAwgZswEAYHKoZIzj0CAQYF
K4EEACMDgYYABAE3rJQRc77YZmgEah4V39BzP4m8WGJNYrfvTcP+Bs70nbmFEnKE
YN4dQcAIwbJyCAgqtgavwniZ12yh4VavrEn9lAAhMN99afkfNFsTEbPC/B/j5J3X
g8N4hyMvvDNMiNZsXqI7nBk2A3Kalhu2O7UoBIszBIN9/DyvS/ouGqGHjq6rMaNx
MG8wDgYDVR0PAQH/BAQDAgKkMB0GA1UdJQQWMBQGCCsGAQUFBwMBBggrBgEFBQcD
AjAPBgNVHRMBAf8EBTADAQH/MB0GA1UdDgQWBBR/aKTqT1BVxqJ3s9N/60F+wMrv
7zAOBgNVHREBAf8EBDACggAwCgYIKoZIzj0EAwQDgYwAMIGIAkIBgDqQS3YiEy/H
HkJb4zmhipVQqkQt0ZpgxZP4WIU5qZfAXowmyG2Q1043fA1tD8pec1DG3hhtaDFi
WxGA3is1oUkCQgCpqiQAqNadTgDeTD56Z07dpkbG2ykivYRd0gxf5KfDVabdtdVc
aAdJSz65XHzUoOQ7kyHec7I3Lvz69VWcIC+CnA==
-----END CERTIFICATE-----
```

mtls-server-ca-cert.pem


```
-----BEGIN CERTIFICATE-----
MIIB+DCCAVmgAwIBAgIRANLSAiRfUiAo9oYpufJbB2YwCgYIKoZIzj0EAwQwADAe
Fw0yNTAyMTMxNDQ3NTJaFw0yNzAyMTMxNDQ3NTJaMAAwgZswEAYHKoZIzj0CAQYF
K4EEACMDgYYABAE3rJQRc77YZmgEah4V39BzP4m8WGJNYrfvTcP+Bs70nbmFEnKE
YN4dQcAIwbJyCAgqtgavwniZ12yh4VavrEn9lAAhMN99afkfNFsTEbPC/B/j5J3X
g8N4hyMvvDNMiNZsXqI7nBk2A3Kalhu2O7UoBIszBIN9/DyvS/ouGqGHjq6rMaNx
MG8wDgYDVR0PAQH/BAQDAgKkMB0GA1UdJQQWMBQGCCsGAQUFBwMBBggrBgEFBQcD
AjAPBgNVHRMBAf8EBTADAQH/MB0GA1UdDgQWBBR/aKTqT1BVxqJ3s9N/60F+wMrv
7zAOBgNVHREBAf8EBDACggAwCgYIKoZIzj0EAwQDgYwAMIGIAkIBgDqQS3YiEy/H
HkJb4zmhipVQqkQt0ZpgxZP4WIU5qZfAXowmyG2Q1043fA1tD8pec1DG3hhtaDFi
WxGA3is1oUkCQgCpqiQAqNadTgDeTD56Z07dpkbG2ykivYRd0gxf5KfDVabdtdVc
aAdJSz65XHzUoOQ7kyHec7I3Lvz69VWcIC+CnA==
-----END CERTIFICATE-----
```


```
-----BEGIN EC PRIVATE KEY-----
MIHcAgEBBEIBCZn46aZgVxvUMHDFiGYaREkEo7L+i6ZSE4SgupGhfaWiMcMAxWKh
Ds/t3PCOKx+19mMDOmb3BCKOXCEL50Td1TqgBwYFK4EEACOhgYkDgYYABAE3rJQR
c77YZmgEah4V39BzP4m8WGJNYrfvTcP+Bs70nbmFEnKEYN4dQcAIwbJyCAgqtgav
wniZ12yh4VavrEn9lAAhMN99afkfNFsTEbPC/B/j5J3Xg8N4hyMvvDNMiNZsXqI7
nBk2A3Kalhu2O7UoBIszBIN9/DyvS/ouGqGHjq6rMQ==
-----END EC PRIVATE KEY-----
```

Sliver C2 – Intercepting mTLS

mitmproxy

127.0.0.1:8081/#/flows/c7c69365-8481-4370-8dd8-1d925e117d3c/request

File | Capture | Flow List | Options | Flow

Search

Highlight

Find

Intercept

Resume All

Intercept

Path	Method	Status	Size	Time
192.168.233.179:50217 ↔ 192.168.233.135:8888	TCP		54b	...
127.0.0.1:50222 ↔ 192.168.233.135:8888	TCP		416b	...
127.0.0.1:50224 ↔ 192.168.233.135:8888	TCP		54b	...
127.0.0.1:50228 ↔ 192.168.233.135:8888	TCP		171b	...
127.0.0.1:50230 ↔ 192.168.233.135:8888	TCP		4.2kb	...
127.0.0.1:50233 ↔ 192.168.233.135:8888	TCP		54b	...
127.0.0.1:50235 ↔ 192.168.233.135:8888	TCP		54b	...
127.0.0.1:50237 ↔ 192.168.233.135:8888	TCP		54b	...
127.0.0.1:50239 ↔ 192.168.233.135:8888	TCP		54b	...

Stream Data | Connection | Timing | Comment

12 Messages

View: auto

2025-10-03 08:18:24.074

```
0000:   64 01 00 00                                      d...
```

2025-10-03 08:18:24.090

```
...\xdf.
.TENDER_BATH..DESKTOP-S0VIANL.$e82c4d56-af7f-983f-a422-39e3dc0b4bc8".DESKTOP-S0VIANL\Ruperto*.S-1-5-21-3414424059-3936699897-4
```

2025-10-03 08:19:06.913

```
0000:   41 00 00 00  08 9d df 94  f2 c6 92 89  d1 f4 01 10   A...............
0010:   05 1a 32 0a  01 2e 4a 2d  10 ff af 9d  c2 df 01 4a   ..2...J-.......J
0020:   24 30 32 38  39 30 30 64  32 2d 33 39  33 32 2d 34   $028900d2-3932-4
0030:   65 61 62 2d  62 36 64 34  2d 30 36 32  61 36 30 37   eab-b6d4-062a607
0040:   36 36 32 61  35                                      662a5
```

2025-10-03 08:19:06.950

```
0000:   f9 00 00 00                                      ....
```

2025-10-03 08:19:06.971

```
.\x9d\xf2\x89\xd1\xf4..\xeb.
1C:\Users\Ruperto\Desktop\Research Binaries\sliver....
.....\x80  \x9a\xf7\xfd\xc6.*
drwxrwxrwx./
.mtls-implant-ca-cert.pem.\xd2. \xfc\xec\xfd\xc6.*
-rw-rw-rw-..
.mtls-server-ca-cert.pem.\xd8. \x8a\xf7\xfd\xc6.*
-rw-rw-rw-.'
.mtlsbeacon.exe.\x80\x90\x93. \x83\x82\xfe\xc6.*
-rw-rw-rw-".PDT(\x90\xbb\xfe\xff\xff\xff\xff\xff\xff.
```

Reverse Proxy

Requests are forwarded to a preconfigured destination.

☑ Forward tls traffic to 192.168.233.135:8888

Reverse proxy to tls://192.168.233.135:8888 listening at *:8888.

Add additional server

Sliver C2 – Intercepting mTLS

```python
import logging

from mitmproxy import tcp
from mitmproxy.utils import strutils


def tcp_message(flow: tcp.TCPFlow):
    message = flow.messages[-1]

    if b"mtlsbeacon.exe" in message.content:
        original = message.content
        message.content = original.replace(b"script.py", b"pwned.exe")

        logging.info("=== TCP Modification Detected ===")
        logging.info(f"From Client: {message.from_client}")
        logging.info(f"Original: {strutils.bytes_to_escaped_str(original)}")
        logging.info(f"Modified: {strutils.bytes_to_escaped_str(message.content)}")
        logging.info("==================================")
    else:
        logging.info(
            f"tcp_message[from_client={message.from_client}), content={strutils.bytes_to_escaped_str(message.content)}]"
        )
```

Sliver C2 – Intercepting mTLS

Desktop › Research Binaries › sliver

Name	Date modified
mtlsbeacon.exe	10/3/2025 2:02 AM
mtlsbeacon_not_obfuscated.exe	10/3/2025 2:03 AM
mtls-implant-ca-cert.pem	10/3/2025 1:59 AM
mtls-server-ca-cert.pem	10/3/2025 1:59 AM
script.py	10/3/2025 2:21 AM

```
sliver (WIDESPREAD_POSSIBILITY) > ls

C:\Users\Ruperto\Desktop\Research Binaries\sliver (6 items, 49.4 MiB)
==========================================================================
drwxrwxrwx  .                                <dir>     Fri Oct 03 02:03:51 -0700 2025
-rw-rw-rw-  mtls-implant-ca-cert.pem         785 B     Fri Oct 03 01:59:30 -0700 2025
-rw-rw-rw-  mtls-server-ca-cert.pem          1.1 KiB   Fri Oct 03 01:59:17 -0700 2025
-rw-rw-rw-  mtlsbeacon.exe                   32.3 MiB  Fri Oct 03 02:02:11 -0700 2025
-rw-rw-rw-  mtlsbeacon_not_obfuscated.exe    17.1 MiB  Fri Oct 03 02:03:25 -0700 2025
-rw-rw-rw-  pwned.exe                        1.3 KiB   Fri Oct 03 02:21:18 -0700 2025
```

Wrapping Up

- Intercepting encrypted traffic is always possible.
- The mind map covers most common scenarios.
- We didn’t cover mTLS interception using Frida with Golang Binaries (guess why)
- On Linux is more or less the same. However, there are some tricks like LD_PRELOAD, not present in Windows.
- Slides + Workshop examples can be found on GitHub.

Further Reading

- Splunk Validated Architectures
- Trellix Agent-Server communication
- SSL Pinning vs SSL Verification
- Pinning Cheat Sheet OWASP
- Golang Bypass SSL Verification
- MITM Proxy Settings
- Understanding Mutual TLS
- TLS.Config Golang
- SSL_Read()
- SSL_Write()
- Sliver C2
- Sliver C2 Forensics
- LD_Preload
- EVP_EncryptInit_ex() & EVP_EncryptUpdate()

Challenge – Intercepting Encrypted payloads

- Your are tasked to get the symmetric key used to encrypt the payload sent by ***encrypted_payload_client.exe*** (on the fly)
- You will be provided with ***encrypted_payload_client.exe*** and ***encrypted_payload_server.exe***. Client connects to port 8080 just once.
- You can use IDA to solve the challenge, but that is not the intended way :(
- Recommendations:
  - Identify the protocol used (HTTP or TCP)
  - Identify the encryption mechanism
  - Identify which tools you should use

Challenge – Solution

- Runnnig the binary we see the following:
  - The binary expects the server's IP
  - It's connecting to port 8080

```
C:\Windows\System32\cmd.exe

C:\Users\Ruperto\Desktop\Research Binaries\tcp and encrypted payload>encrypted_packet_client.exe 192.168.233.1
Server says: Message received

C:\Users\Ruperto\Desktop\Research Binaries\tcp and encrypted payload>
```

```
C:\Users\Jorge\Downloads\TEST>encrypted_packet_server.exe
TCP Server listening on port 8080...
Received: Hello from client
```

Challenge – Solution

- Eavesdropping the traffic, we identify TCP packets with encrypted data sent.

tcp.stream eq 5

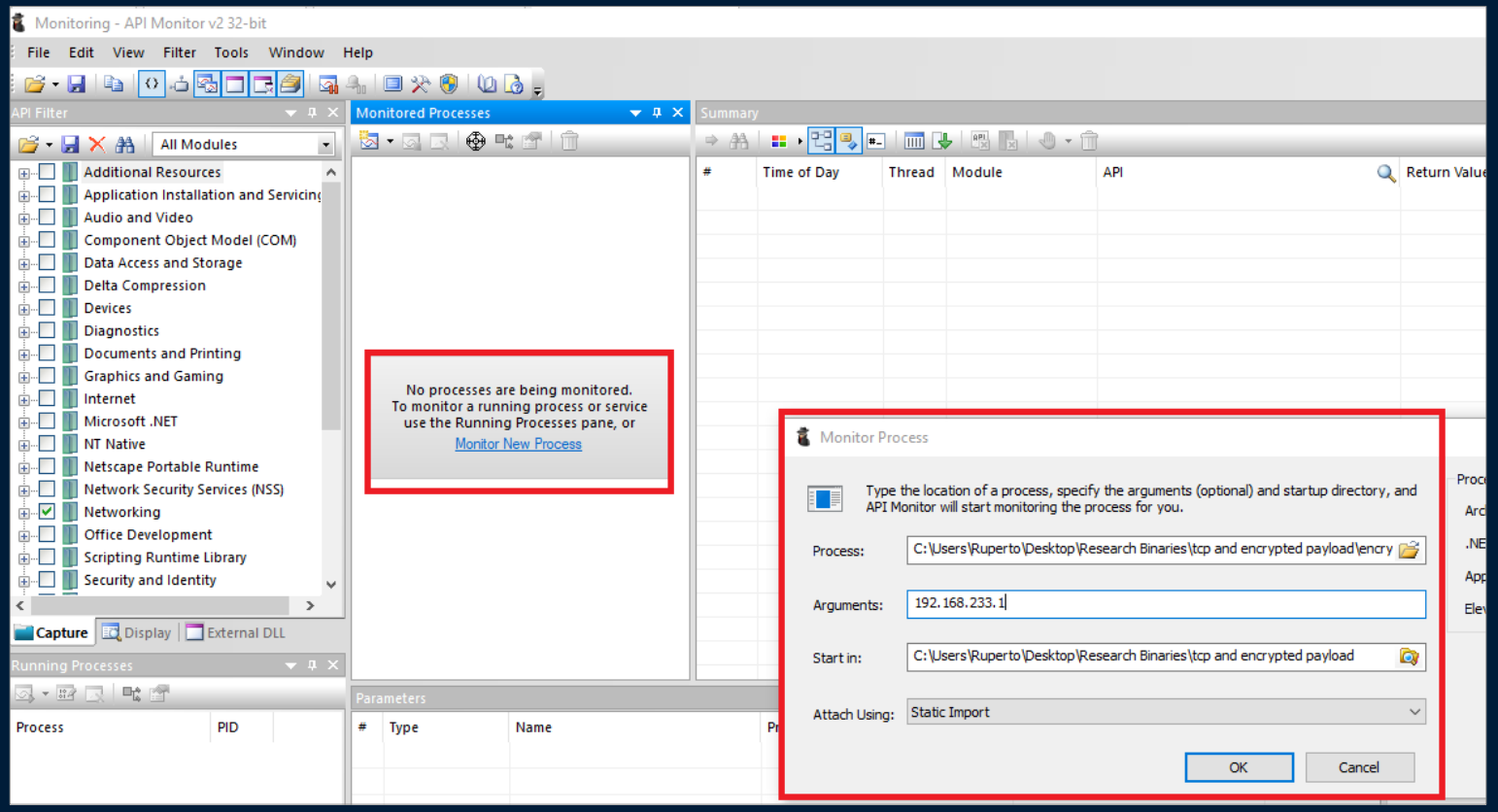
No.	Time	Source	Destination	Protocol	Length	Info
856	4.364509	192.168.233.179	192.168.233.1	TCP	66	49940 → 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
857	4.366140	192.168.233.1	192.168.233.179	TCP	66	8080 → 49940 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
858	4.366808	192.168.233.179	192.168.233.1	TCP	54	49940 → 8080 [ACK] Seq=1 Ack=1 Win=262656 Len=0
859	4.420836	192.168.233.179	192.168.233.1	TCP	86	49940 → 8080 [PSH, ACK] Seq=1 Ack=1 Win=262656 Len=32
860	4.423167	192.168.233.1	192.168.233.179	TCP	86	8080 → 49940 [PSH, ACK] Seq=1 Ack=33 Win=65280 Len=32
861	4.423195	192.168.233.1	192.168.233.179	TCP	54	8080 → 49940 [FIN, ACK] Seq=33 Ack=33 Win=65280 Len=0
862	4.423586	192.168.233.179	192.168.233.1	TCP	54	49940 → 8080 [ACK] Seq=33 Ack=34 Win=262656 Len=0
863	4.424143	192.168.233.179	192.168.233.1	TCP	54	49940 → 8080 [FIN, ACK] Seq=33 Ack=34 Win=262656 Len=0
864	4.426482	192.168.233.1	192.168.233.179	TCP	54	8080 → 49940 [ACK] Seq=34 Ack=34 Win=65280 Len=0

Wireshark · Seguir secuencia TCP (tcp.stream eq 5) · VMware Network Adapter VMnet8

```
d^............f.K&.B@.o...}..ja
...7.0...U..Z.^.j..2...g......|p
```

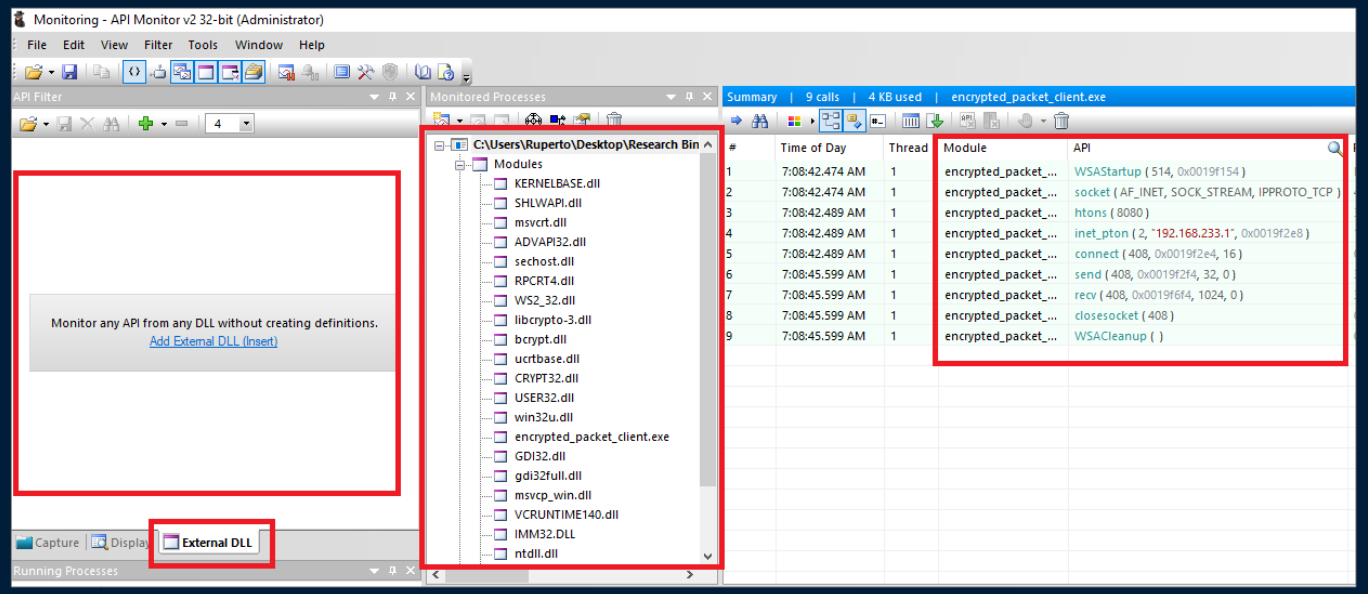
Challenge – Solution

- We can use API Monitor to get a better idea of which APIs are been used.
- Notice that Networking is the only module enabled.



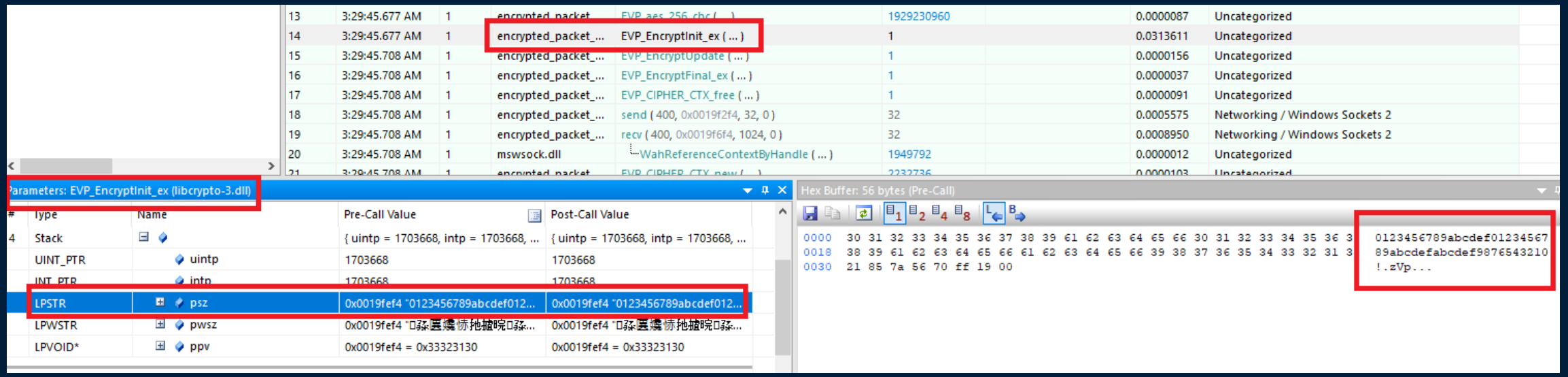
Challenge – Solution

- As we only selected Networking API calls, the output is extremely poor.
- As there are some DLLs loaded by the binary, we can select the ones related to cryptography to get more functions like:
  - Bcrypt.dll
  - Libcrypto-3.dll
  - Crypt32.dll



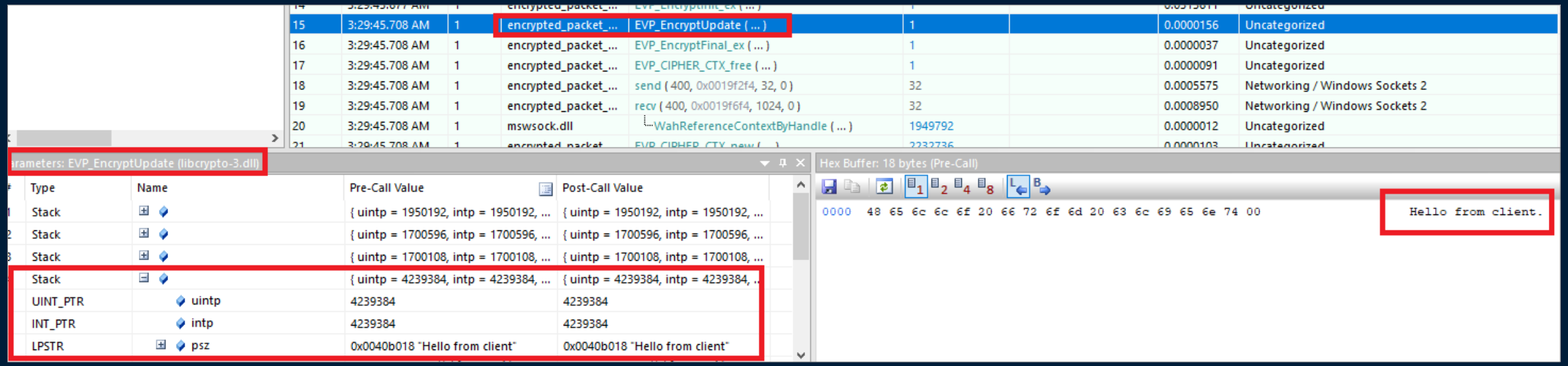
Challenge – Solution

- EVP_EncryptInit_ex() – The encrypted key and the IV are passed as the forth parameter. We can find it using API Monitor as expected.



Challenge – Solution

- EVP_EncryptUpdate(): Cleartext traffic is passed as the forth parameter too.



Challenge – Solution

- With that information, we can create a Frida script to extract the data automatically.

- Note: we can use Frida Trace to automatically detect all the API calls invoked by the binary. However, for the sake of simplicity, we used API Monitor.

```
const moduleName = "libcrypto-3.dll";  // Change if needed
const functionName = "EVP_EncryptInit_ex";

// Locate the symbol using Frida v17 syntax
const target = Process.getModuleByName(moduleName).getExportByName(functionName);

//int EVP_EncryptInit_ex(EVP_CIPHER_CTX *ctx, const EVP_CIPHER *type, ENGINE *impl, unsigned char *key, unsigned
char *iv);
//EVP_EncryptInit_ex() sets up cipher context ctx for encryption with cipher type from ENGINE impl. ctx must be
initialized before calling this function. key is the symmetric key to use and iv is the IV to use (if necessary).
//In other words, we want the key and the iv.

Interceptor.attach(target, {
  onEnter(args) {
    try {
      const key = args[3].readByteArray(32); // adjust length as needed
      console.log("key:", key);
    } catch (e) {
      console.log("Failed to read key:", e);
    }

    // unsigned char *iv
    //console.log("iv ptr:", args[4]);
    try {
      const iv = args[4].readByteArray(16); // adjust length as needed
      console.log("iv:", iv);
    } catch (e) {
      console.log("Failed to read iv:", e);
    }
  }
});
```

Challenge – Solution

- The key extracted is (bear in mind that its length is 32): **0123456789ABCDEF0123456789ABCDEF**.
- The IV extracted is: **abcdef9876543210**

```
C:\Users\Ruperto\Desktop\Research Binaries\tcp and encrypted payload>frida -f encrypted_packet_client.exe 192.168.233.1 -l hook.js
     ____
    / _  |   Frida 17.2.12 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Local System (id=local)
Spawned `encrypted_packet_client.exe 192.168.233.1`. Resuming main thread!
key:            0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
00000000  30 31 32 33 34 35 36 37 38 39 61 62 63 64 65 66  0123456789abcdef
00000010  30 31 32 33 34 35 36 37 38 39 61 62 63 64 65 66  0123456789abcdef
iv:            0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
00000000  61 62 63 64 65 66 39 38 37 36 35 34 33 32 31 30  abcdef9876543210
Server says: Message received
Process terminated
[Local::encrypted_packet_client.exe ]->

Thank you for using Frida!
```

Challenge – Solution

- With this other script we can intercept the information sent by the binary before is encrypted.
- We can even merge both scripts.

```
const moduleName = "libcrypto-3.dll";  // Adjust if needed
const functionName = "EVP_EncryptUpdate";

// Locate the symbol using Frida v17 syntax
const target = Process.getModuleByName(moduleName).getExportByName(functionName);

//int EVP_EncryptUpdate(EVP_CIPHER_CTX *ctx, unsigned char *out, int *outl, unsigned char *in, int inl);
//EVP_EncryptUpdate() encrypts inl bytes from the buffer in and writes the encrypted version to out. In other
words, in contains the message and inl is its length.

Interceptor.attach(target, {
  onEnter(args) {

    try {
      const inl = args[4].toInt32(); // int inl
      const inputData = args[3].readByteArray(inl);
      console.log("in data:", inputData);
    } catch (e) {
      console.log("Failed to read input data:", e);
    }

    // int inl
    console.log("inl:", args[4].toInt32());
  }
});
```

Challenge – Solution

- The message intercepted is: **Hello from Client**

```
C:\Users\Ruperto\Desktop\Research Binaries\tcp and encrypted payload>frida -f encrypted_packet_client.exe 192.168.233.1 -l get_cleartext.js
     ____
    / _  |   Frida 17.2.12 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Local System (id=local)
Spawned `encrypted_packet_client.exe 192.168.233.1`. Resuming main thread!
Server says: Message received
in data:            0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
00000000  48 65 6c 6c 6f 20 66 72 6f 6d 20 63 6c 69 65 6e  Hello from clien
00000010  74                                               t
[Local::encrypted_packet_client.exe ]-> inl: 17
Process terminated
[Local::encrypted_packet_client.exe ]->

Thank you for using Frida!
```